



DATA STRUCTURES

โครงสร้างข้อมูล

Struct · Linked list · Stack · Queue · Enum · Union · File

สอน. ค่าย 1 วิชาคอมพิวเตอร์

ศูนย์โรงเรียนสวนกุหลาบวิทยาลัย นนทบุรี



ข้อมูลชุดเดียวกัน อาจมีกฏการนำออกต่างกัน

เรานำข้อมูล 10, 20 และ 30 เข้ามาตามลำดับ ถ้าต้องนำข้อมูลออกหนึ่งตัว คำตอบจำเป็นต้องเป็น 10 เสมอหรือไม่

ลองคิดถึงสองสถานการณ์: คนที่รอนานที่สุดควรได้รับบริการก่อน แต่เมื่อกดยอนการทำงาน เราต้องยอนสิ่งที่เพิ่งทำล่าสุดก่อน

10

เข้ามาลำดับที่ 1

20

เข้ามาลำดับที่ 2

30

เข้ามาลำดับที่ 3



บทวน Pointer ก่อนเชื่อมข้อมูล

หลังทำคำสั่งทั้งสองบรรทัดท้ายแล้ว x และ y มีค่าเท่าใด และ p ชี้ไปยังตัวแปรใด

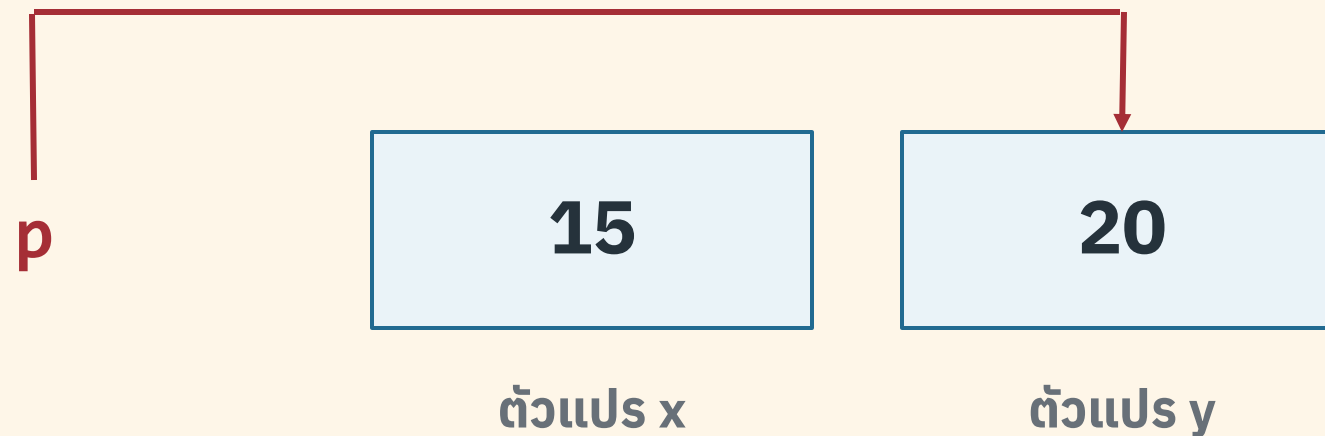
```
int x = 10;  
int y = 20;  
int* p = &x;  
*p = 15;  
p = &y;
```



การเปลี่ยนค่าและการเปลี่ยนตัวชี้ต่างกัน

คำสั่ง $*p = 15$ เปลี่ยนค่าของ x เพราะขณะนั้น p ชี้ไปยัง x ส่วนคำสั่ง $p = \&y$ เปลี่ยนให้ p ไปชี้ y โดยค่าของ y ยังเป็น 20

แนวคิดนี้จะใช้กับ linked list เช่นกัน บางคำสั่งเลื่อนตัวเดิน แต่บางคำสั่งเปลี่ยนการเชื่อมโยงของข้อมูล





Struct



บุคคล

- ชื่อ
- เบอร์โทรศัพท์
- อีเมล



Struct



นักศึกษา

- รหัสนักศึกษา
- ชื่อนักศึกษา
- เกรด



Struct



วันที่

- วัน

- เดือน

- ปี



Struct



เวลา

- ชั่วโมง
- นาที
- วินาที



Struct

- คือกลุ่มของตัวแปรหลายประเภท
รวมตัวกันเป็นประเภทข้อมูลใหม่ประเภทหนึ่ง
- ในฐานข้อมูล เราจะเรียกข้อมูลที่เก็บไว้ว่า ระเบียน (record) ซึ่งประกอบด้วยหลายๆ ฟิลด์ (field) ตัวอย่างเช่น record ของนักศึกษา
- ถูกใช้เพื่อรวมประเภทข้อมูลที่แตกต่างกัน เช่นเดียวกับที่อาร์เรย์ใช้ในการรวมประเภทข้อมูลที่เหมือนกัน
- จะถูกประกาศโดยใช้ “struct”

field

รหัสนักศึกษา

ชื่อนักศึกษา

เกรด





Struct ช่วยให้ข้อมูลที่เกี่ยวข้องอยู่ด้วยกัน

ถ้าเราต้องเก็บเวลาสองชุด การใช้ h1, m1, s1 และ h2, m2, s2 ทำให้ต้องคอยจำว่าตัวแปรใดเป็นของชุดเดียวกัน

เราสามารถสร้างชนิดข้อมูล Time เพื่อรวมชั่วโมง นาที และวินาทีไว้ด้วยกันได้ จากนั้นตัวแปรหนึ่งตัวก็แทนเวลาได้หนึ่งชุด

Time start

9

start.h

30

start.m

0

start.s



การประกาศชนิดข้อมูลและตัวแปร Struct

คำประกาศ struct Time กำหนดว่าเวลาหนึ่ง
ชุดมีข้อมูลอะไรบ้าง ส่วน Time start สร้าง
ตัวแปรที่ใช้เก็บเวลาจริงหนึ่งชุด

ในตัวอย่างนี้ start แทนเวลา 9 นาฬิกา 30 นาที
0 วินาที สมาชิกของ struct อาจเป็นชนิด
เดียวกันหรือต่างชนิดกันก็ได้

```
struct Time {  
    int h;  
    int m;  
    int s;  
};  
  
Time start{9, 30, 0};
```



เราใช้จุดเพื่อเข้าถึงสมาชิกของ Struct

เมื่อรู้จักตัวแปร start แล้ว เราใช้ start.h เพื่อเข้าถึงชั่วโมง และใช้ start.m เพื่อเข้าถึงนาที

ถ้าสั่ง start.m = 45 จะเปลี่ยนเฉพาะนาทีของ start ส่วนชั่วโมงและวินาทียังคงเดิม ลองทายว่าเวลาหลังคำสั่งจะเป็นเท่าใด

```
Time start{9, 30, 0};
```

```
start.m = 45;
```

```
std::cout << start.h << ":"  
          << start.m;
```



เราส่ง Struct ให้ฟังก์ชันได้ทั้งชุด

ฟังก์ชันสามารถรับ Time ได้โดยตรง จึงไม่ต้องส่งชั่วโมง นาที และวินาทีแยกเป็นพารามิเตอร์หลายตัว

ตัวอย่างทางขวารับสำเนาของเวลา ส่วนพารามิเตอร์แบบ Time& จะอ้างถึงตัวแปรของผู้เรียกและใช้แก้ค่าเดิมได้ เหมือน reference ที่เคยเรียนมา

```
void show_hour(Time t) {  
    std::cout << t.h;  
}
```

```
Time start{9, 30, 0};  
show_hour(start);
```



Struct หนึ่งตัวมี Struct อีกตัวเป็นสมาชิกได้

หากข้อมูลบุคคลมีที่อยู่ เราสามารถสร้าง struct Address แล้วนำมาเป็นสมาชิกของ struct Person ได้

การเข้าถึงสมาชิกซ้อนกันใช้จุดต่อกัน เช่น person.address.city ซึ่งอ่านได้ว่าเมืองในที่อยู่ของบุคคลนี้

```
struct Address {  
    std::string city;  
    std::string road;  
};  
struct Person {  
    std::string name;  
    int age;  
    Address address;  
};
```



แบบไม่ใช้ struct

```
#include <iostream>
#include <cmath>

int main() {
    int h1, h2, m1, m2, s1, s2;

    std::cout << "Time 1\n";
    std::cout << "    hour: ";
    std::cin >> h1;
    std::cout << "    minute: ";
    std::cin >> m1;
    std::cout << "    second: ";
    std::cin >> s1;

    std::cout << "Time 2\n";
    std::cout << "    hour: ";
    std::cin >> h2;
    std::cout << "    minute: ";
    std::cin >> m2;
    std::cout << "    second: ";
    std::cin >> s2;

    int seconds1 = h1 * 3600 + m1 * 60 + s1;
    int seconds2 = h2 * 3600 + m2 * 60 + s2;
    int diff = std::abs(seconds1 - seconds2);

    std::cout << "Seconds difference = " << diff <<
    std::endl;

    return 0;
}
```

แบบใช้ struct

```
#include <iostream>
#include <cmath>

struct Time {
    int h, m, s;
};

void readTime(Time& t) {
    std::cout << "    hour: ";
    std::cin >> t.h;
    std::cout << "    minute: ";
    std::cin >> t.m;
    std::cout << "    second: ";
    std::cin >> t.s;
}

int timeDiff(const Time& t1, const Time& t2) {
    int seconds1 = t1.h * 3600 + t1.m * 60 + t1.s;
    int seconds2 = t2.h * 3600 + t2.m * 60 + t2.s;
    return std::abs(seconds1 - seconds2);
}

int main() {
    Time t1, t2;

    std::cout << "Time 1\n";
    readTime(t1);
    std::cout << "Time 2\n";
    readTime(t2);

    int diff = timeDiff(t1, t2);

    std::cout << "Seconds difference = " << diff << std::endl;

    return 0;
}
```



แบบไม่ใช้ struct

```
#include <iostream>
#include <cmath>
```

```
int main() {
    int h1, h2, m1, m2, s1, s2;
```

```
    std::cout << "Time 1\n";
    std::cout << "    hour: ";
    std::cin >> h1;
    std::cout << "    minute: ";
    std::cin >> m1;
    std::cout << "    second: ";
    std::cin >> s1;
```

```
    std::cout << "Time 2\n";
    std::cout << "    hour: ";
    std::cin >> h2;
    std::cout << "    minute: ";
    std::cin >> m2;
    std::cout << "    second: ";
    std::cin >> s2;
```

```
    int seconds1 = h1 * 3600 + m1 * 60 + s1;
    int seconds2 = h2 * 3600 + m2 * 60 + s2;
    int diff = std::abs(seconds1 - seconds2);
```

```
    std::cout << "Seconds difference = " << diff <<
    std::endl;
```

```
    return 0;
}
```

แบบไม่ใช้ struct:

- ตัวแปร h1, m1, s1 สำหรับเวลา 1 และ h2, m2, s2 สำหรับเวลา 2 ถูกประกาศแยกกัน ซึ่งอาจทำให้ยุ่งยากเมื่อจัดการกับค่าของเวลาหลาย ๆ ชุด
- การคำนวณทำได้โดยการเข้าถึงตัวแปรแต่ละตัวโดยตรง

```
#include <iostream>
```

```
int seconds1 = t1.h * 3600 + t1.m * 60 + t1.s;
int seconds2 = t2.h * 3600 + t2.m * 60 + t2.s;
return std::abs(seconds1 - seconds2);
}
```

```
int main() {
    Time t1, t2;
```

```
    std::cout << "Time 1\n";
    readTime(t1);
    std::cout << "Time 2\n";
    readTime(t2);
```

```
    int diff = timeDiff(t1, t2);
```

```
    std::cout << "Seconds difference = " << diff << std::endl;

    return 0;
}
```



แบบใช้ struct:

- struct Time จะรวบรวมชั่วโมง, นาที และวินาทีเข้าเป็นหน่วยเดียว ซึ่งทำให้โค้ดสะอาดขึ้น โดยเฉพาะเมื่อต้องจัดการกับค่าของเวลาหลาย ๆ ชุด
- ฟังก์ชัน readTime และ timeDiff ทำงานกับวัตถุประเภท Time ทำให้โค้ดเป็นแบบโมดูลาร์มากขึ้นและง่ายต่อการดูแล
- การใช้ struct ทำให้โค้ดมีระเบียบและเป็นระบบมากขึ้น ทำให้ง่ายต่อการจัดการเมื่อโค้ดขยายตัวหรือเมื่อจัดการกับสถานการณ์ที่ซับซ้อนมากขึ้น

```
std::endl;
```

```
return 0;
```

```
}
```

```
#include <iostream>
```

```
#include <cmath>
```

```
struct Time {  
    int h, m, s;  
};
```

```
void readTime(Time& t) {  
    std::cout << " hour: ";  
    std::cin >> t.h;  
    std::cout << " minute: ";  
    std::cin >> t.m;  
    std::cout << " second: ";  
    std::cin >> t.s;  
}
```

```
int timeDiff(const Time& t1, const Time& t2) {  
    int seconds1 = t1.h * 3600 + t1.m * 60 + t1.s;  
    int seconds2 = t2.h * 3600 + t2.m * 60 + t2.s;  
    return std::abs(seconds1 - seconds2);  
}
```

```
int main() {  
    Time t1, t2;
```

```
    std::cout << "Time 1\n";  
    readTime(t1);  
    std::cout << "Time 2\n";  
    readTime(t2);
```

```
    int diff = timeDiff(t1, t2);
```

```
    std::cout << "Seconds difference = " << diff << std::endl;
```

```
    return 0;
```

```
}
```



แบบไม่ใช้ struct

```
#include <iostream>
#include <cmath>

int main() {
    int h1, h2, m1, m2, s1, s2;

    std::cout << "Time 1\n";
    std::cout << "    hour: ";
    std::cin >> h1;
    std::cout << "    minute: ";
    std::cin >> m1;
    std::cout << "    second: ";
    std::cin >> s1;

    std::cout << "Time 2\n";
    std::cout << "    hour: ";
    std::cin >> h2;
    std::cout << "    minute: ";
    std::cin >> m2;
    std::cout << "    second: ";
    std::cin >> s2;

    int seconds1 = h1 * 3600 + m1 * 60 + s1;
    int seconds2 = h2 * 3600 + m2 * 60 + s2;
```

```
    return 0;
}
```

แบบใช้ struct

```
#include <iostream>
#include <cmath>

struct Time {
    int h, m, s;
};

void readTime(Time& t) {
    std::cout << "    hour: ";
    std::cin >> t.h;
    std::cout << "    minute: ";
    std::cin >> t.m;
    std::cout << "    second: ";
    std::cin >> t.s;
}

int timeDiff(const Time& t1, const Time& t2) {
    int seconds1 = t1.h * 3600 + t1.m * 60 + t1.s;
    int seconds2 = t2.h * 3600 + t2.m * 60 + t2.s;
    return std::abs(seconds1 - seconds2);
}

int main() {
    Time t1, t2;

    std::cout << "Time 1\n";
    readTime(t1);
    std::cout << "Time 2\n";
```

```
    return 0;
}
```

<https://github.com/hchaser/posn66>



แบบไม่ใช้ struct

```
#include <iostream>
#include <cmath>

int main() {
    int h1, h2, m1, m2, s1, s2;

    std::cout << "Time 1\n";
    std::cout << "    hour: ";
    std::cin >> h1;
    std::cout << "    minute: ";
    std::cin >> m1;
    std::cout << "    second: ";
    std::cin >> s1;

    std::cout << "Time 2\n";
    std::cout << "    hour: ";
    std::cin >> h2;
    std::cout << "    minute: ";
    std::cin >> m2;
    std::cout << "    second: ";
    std::cin >> s2;

    int seconds1 = h1 * 3600 + m1 * 60 + s1;
    int seconds2 = h2 * 3600 + m2 * 60 + s2;
```

```
    return 0;
}
```

แบบใช้ struct

```
#include <iostream>
#include <cmath>

struct Time {
    int h, m, s;
};

void readTime(Time& t) {
    std::cout << "    hour: ";
    std::cin >> t.h;
    std::cout << "    minute: ";
    std::cin >> t.m;
    std::cout << "    second: ";
    std::cin >> t.s;
}

int timeDiff(const Time& t1, const Time& t2) {
    int seconds1 = t1.h * 3600 + t1.m * 60 + t1.s;
    int seconds2 = t2.h * 3600 + t2.m * 60 + t2.s;
    return std::abs(seconds1 - seconds2);
}

int main() {
    Time t1, t2;

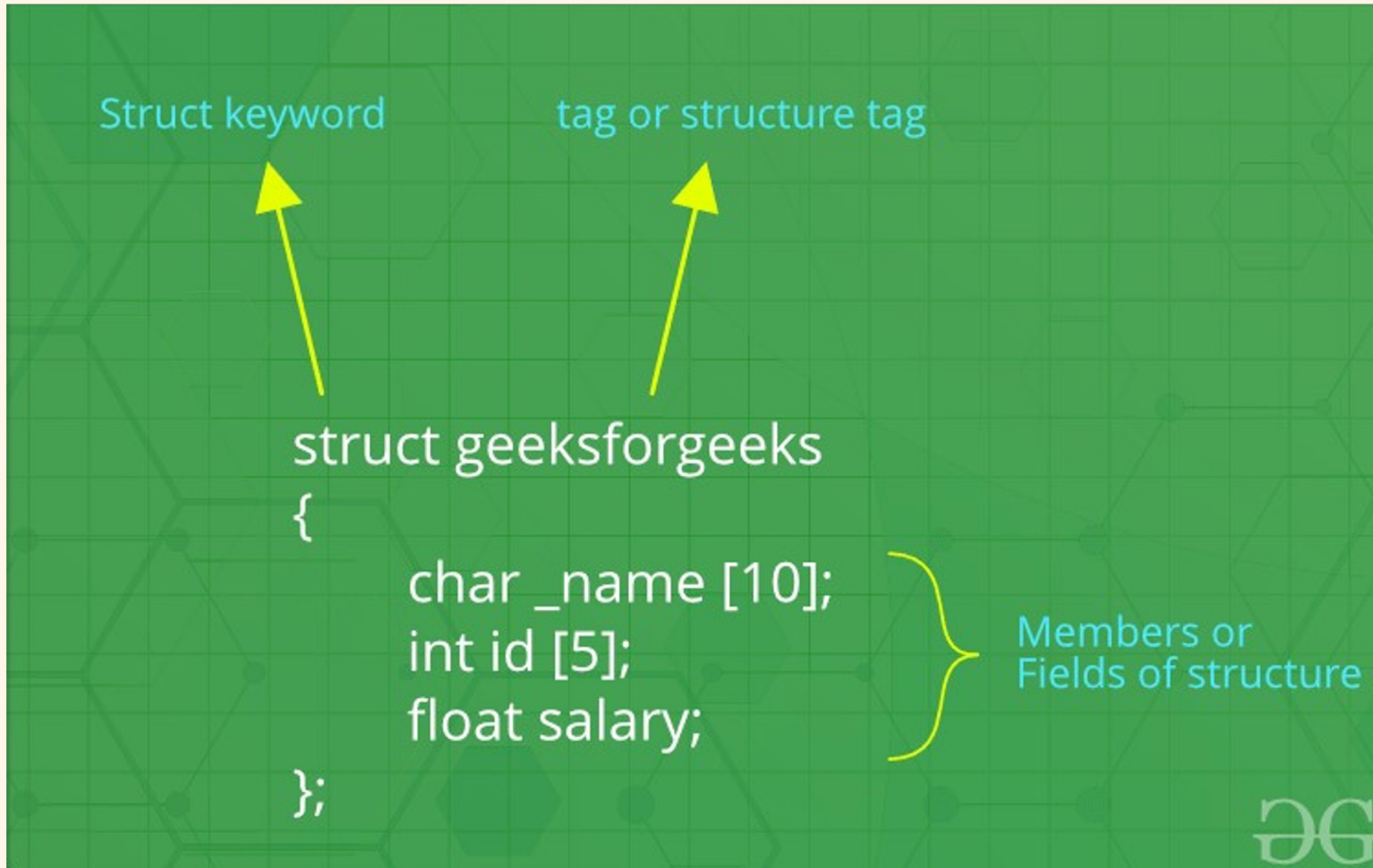
    std::cout << "Time 1\n";
    readTime(t1);
    std::cout << "Time 2\n";
```

```
    return 0;
}
```

<https://github.com/hchaser/posn66>



Structures

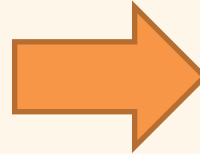


```
struct ชื่อสตรักเจอร์  
{  
    ชนิดข้อมูล ชื่อรายการ;  
    ชนิดข้อมูล ชื่อรายการ;  
    ชนิดข้อมูล ชื่อรายการ;  
    ชนิดข้อมูล ชื่อรายการ;  
};
```



Syntax of Struct

```
struct structureName {  
    dataType member1;  
    dataType member2;  
    ...  
};
```



```
struct Person {  
    char name[50];  
    int id[5];  
    float salary;  
};
```



```
#include <iostream>
#include <string>
using namespace std;

struct Student {
    int ID;        // เก็บ ID นักเรียน
    string name;   // เก็บชื่อของนักเรียน
    float score;   // เก็บคะแนนของนักเรียน
};
```

```
Student ID : 12345
Student Name :
Kongkiat Jedipharm
Student Score : 88.5
```

```
int main()
{
    // ประกาศตัวแปร struct นักเรียน
    Student student1;

    // กำหนดค่าข้อมูลให้กับตัวแปร student1
    student1.ID = 12345;
    student1.name = "Kongkiat Jedipharm";
    student1.score = 88.5;

    // แสดงข้อมูลของนักเรียน
    cout << "Student ID : " << student1.ID << endl;
    cout << "Student Name : " << student1.name << endl;
    cout << "Student Score : " << student1.score << endl;

    return 0;
}
```



Enum ทำให้ชื่อสถานะสื่อความหมาย

ถ้าใช้เลข 0, 1 และ 2 แทนสถานะ เราต้องจำความหมายของแต่ละเลขตลอดเวลา enum ช่วยให้ตั้งชื่อสถานะไว้ในโปรแกรมได้

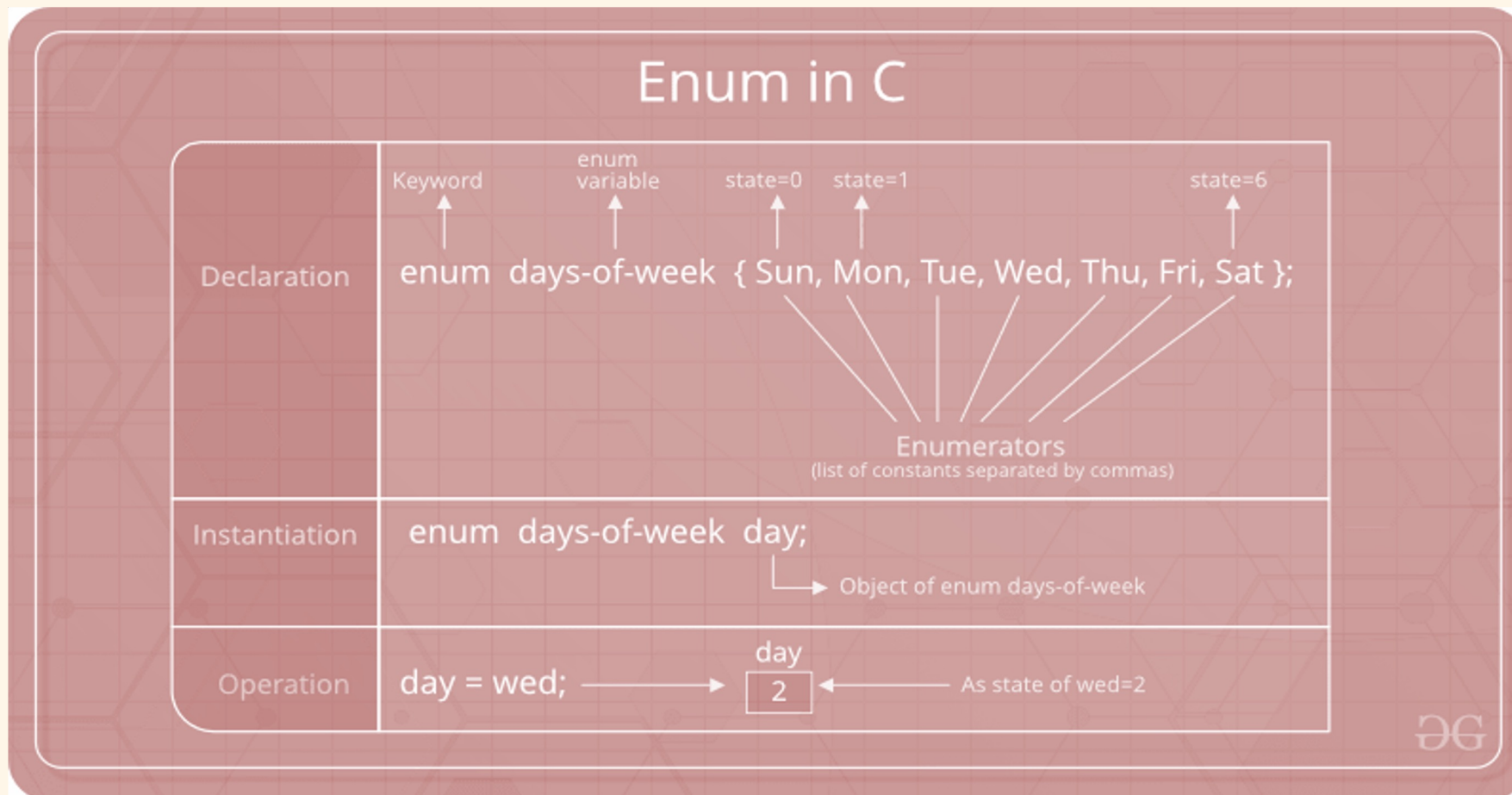
ในบทนี้เราจะใช้ enum class เช่น `Status::Waiting` เพื่อบอกชัดว่าค่านี้เป็นสถานะที่กำลังรอ ไม่ใช่ตัวเลขทั่วไป

```
enum class Status {  
    Waiting, Running, Done  
};  
  
Status state = Status::Waiting;  
state = Status::Running;
```



Enumeration

- Enums คือชนิดข้อมูลที่กำหนดขึ้นเองโดยผู้ใช้ ซึ่งประกอบไปด้วยค่าคงที่แบบจำนวนเต็มที่มีชื่อกำกับ
- Enums ช่วยในการกำหนดค่าคงที่ให้กับชุดของชื่อ เพื่อให้โปรแกรมอ่านง่ายขึ้น, ดูแลรักษาง่ายขึ้น, และเข้าใจได้ง่ายขึ้น
- การประกาศ Enums ใช้คีย์เวิร์ด "enum"





Enumeration

- Enums คือชนิดข้อมูลที่กำหนดขึ้นเองโดยผู้ใช้ ซึ่งประกอบไปด้วยค่าคงที่แบบจำนวนเต็มที่มีชื่อกำกับ
- Enums ช่วยในการกำหนดค่าคงที่ให้กับชุดของชื่อ เพื่อให้โปรแกรมอ่านง่ายขึ้น, ดูแลรักษาง่ายขึ้น, และเข้าใจได้ง่ายขึ้น
- การประกาศ Enums ใช้คีย์เวิร์ด "enum"

```
#include <iostream>
using namespace std;
```

```
enum Status { Status1, Status2,
Status3 };
```

```
Status S1 = Status1;
Status S2 = Status2;
Status S3 = Status3;
```

```
int main()
{
    cout << "The numerical value "
        << "assigned to Status1 : " << S1 << endl;

    cout << "The numerical value "
        << "assigned to Status2 : " << S2 << endl;

    cout << "The numerical value "
        << "assigned to Status3 : " << S3 << endl;

    return 0;
}
```



Enumeration

- Enums คือชนิดข้อมูลที่กำหนดขึ้นเองโดยผู้ใช้ ซึ่งประกอบไปด้วยค่าคงที่แบบจำนวนเต็มที่มีชื่อกำกับ
- Enums ช่วยในการกำหนดค่าคงที่ให้กับชุดของชื่อ เพื่อให้โปรแกรมอ่านง่ายขึ้น, ดูแลรักษาง่ายขึ้น, และเข้าใจได้ง่ายขึ้น
- การประกาศ Enums ใช้คีย์เวิร์ด "enum"

```
#include <iostream>
using namespace std;
```

```
enum Status { Status1, Status2,
Status3 };
```

```
Status S1 = Status1;
```

```
int main()
{
    cout << "The numerical value "
        << "assigned to Status1 : " << S1 << endl;

    cout << "The numerical value "
        << "assigned to Status2 : " << S2 << endl;

    cout << "The numerical value "
        << "assigned to Status3 : " << S3 << endl;
```

The numerical value assigned to Status1 : 0

The numerical value assigned to Status2 : 1

The numerical value assigned to Status3 : 2



Enumeration

ทดสอบเขียน **enumeration**

```
#include <iostream>

enum year { Jan = 1, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec };

int main() {
    for (int i = Jan; i <= Dec; i++) {
        std::cout << i << " ";
    }

    return 0;
}
```

C:\Users\Ohayou\OneDrive\Desktop\pointer\enum2.exe

1 2 3 4 5 6 7 8 9 10 11 12

Process exited after 0.04197 seconds with return value 0

Press any key to continue . . .



ชื่ออีกชื่อหนึ่งของชนิดข้อมูล

ในภาษา C++ เราเขียน Time start; ได้หลังประกาศ struct Time แล้ว โดยไม่ต้องใช้ typedef เพิ่ม

หากต้องการตั้งชื่ออีกชื่อให้ชนิดเดิม เราใช้ using หรือ typedef ได้ ชื่อใหม่นี้ยังหมายถึงชนิดข้อมูลเดียวกัน

```
using ClockTime = Time;  
ClockTime alarm{6, 30, 0};
```

```
// Another alias syntax  
typedef Time TimeOfDay;
```



Challenge 1: เวลาห่างกันกี่วินาที

กำหนดเวลาในวันเดียวกันสองชุด โดยแต่ละชุดมีชั่วโมง นาที และวินาที ให้นักเรียนหาว่าเวลาทั้งสองห่างกันกี่วินาที โดยคำตอบต้องไม่ติดลบ

ให้ใช้ struct Time แทนเวลา และสร้างฟังก์ชันแปลงเวลาเป็นวินาที จากนั้นเขียนฟังก์ชันหาผลต่างระหว่างเวลาสองชุด

12:59:59

เวลาชุดแรก

13:00:00

เวลาชุดที่สอง

ห่างกัน 1 วินาที



ขอบเขตและแนวทางเริ่มต้นของ Challenge 1

ชั่วโมงอยู่ในช่วง 0–23 ส่วนนาทีและวินาทีอยู่ในช่วง 0–59 หากเวลาทั้งสองเท่ากัน คำตอบต้องเป็นศูนย์ ให้เขียน `int to_seconds(Time t)` และ `int difference(Time a, Time b)` เพื่อคืนผลลัพธ์ โจทย์นี้เปรียบเทียบเวลาในวันเดียวกัน จึงไม่คิดช่วงที่เดินข้ามเที่ยงคืน

เวลาชุดแรก	เวลาชุดที่สอง	ผลต่าง
12:59:59	13:00:00	1 วินาที
08:10:20	08:10:20	0 วินาที
00:00:00	23:59:59	86399 วินาที



DATA STRUCTURES

โครงสร้างข้อมูล



DATA STRUCTURES

โครงสร้างข้อมูล

คือวิธีการจัดเก็บและจัดระเบียบข้อมูลภายในคอมพิวเตอร์ โดยมีวัตถุประสงค์หลักเพื่อให้สามารถเข้าถึงและแก้ไขข้อมูลได้อย่างมีประสิทธิภาพ



ปัจจัยในการเลือกใช้โครงสร้างข้อมูล

การเลือกโครงสร้างข้อมูลที่เหมาะสมต้องพิจารณาจากปัจจัยหลัก 3 ประการ ได้แก่

- ลักษณะของการทำงานข้อมูล
- ปริมาณข้อมูลที่จะจัดเก็บ
- ความถี่ในการใช้งานข้อมูล



ประโยชน์และการนำไปประยุกต์ใช้

- การจัดการหน่วยความจำ: การเลือกโครงสร้างข้อมูลที่เหมาะสมช่วยให้ใช้พื้นที่ในหน่วยความจำ (RAM) เกิดประโยชน์สูงสุด
- การจัดการข้อมูลขนาดใหญ่: มีบทบาทสำคัญในการจัดการชุดข้อมูลขนาดใหญ่ และการสร้างดัชนีข้อมูล (Indexing) เพื่อความรวดเร็วในการค้นหา
- การประมวลผลและคำนวณ: เป็นรากฐานสำคัญในการทำงานของอัลกอริทึม (Algorithm) รวมถึงการคำนวณหาเส้นทางในการนำทางต่างๆ



① PRIMITIVE DATA STRUCTURE

โครงสร้างข้อมูลชนิดพื้นฐาน

- โครงสร้างข้อมูลหลักที่ทำงานโดยตรงกับคำสั่งระดับคอมพิวเตอร์
- ใช้กับการดึงและเปลี่ยนแปลงค่าข้อมูลชนิดต่าง ๆ
- มีประสิทธิภาพกับข้อมูลที่มีขนาดเล็ก

integer

จำนวนเต็ม

float

จำนวนจริง

character

ตัวอักษร

boolean

ตรรกะ



② NON-PRIMITIVE DATA STRUCTURE

โครงสร้างข้อมูลชนิดไม่ใช่พื้นฐาน

- ซับซ้อนมากกว่าโครงสร้างข้อมูลชนิดพื้นฐาน
- นิยมใช้เก็บข้อมูลขนาดใหญ่ที่มีความเชื่อมโยงกัน

Linear Data Structure

โครงสร้างข้อมูลเชิงเส้น

เป็นการจัดเรียงข้อมูลในลำดับ

การเข้าถึงข้อมูลทำได้โดยเริ่มจากจุดแรกไล่ไปที่ละลำดับตามโครงสร้าง

Arrays, Linked lists, Stacks, Queues

Non-Linear Data Structure

โครงสร้างข้อมูลไม่เชิงเส้น

การจัดเรียงข้อมูลอย่างไม่มีลำดับ

แต่นั่นความสัมพันธ์แบบลำดับชั้นหรือการเชื่อมโยงที่ซับซ้อน

Trees, Graphs

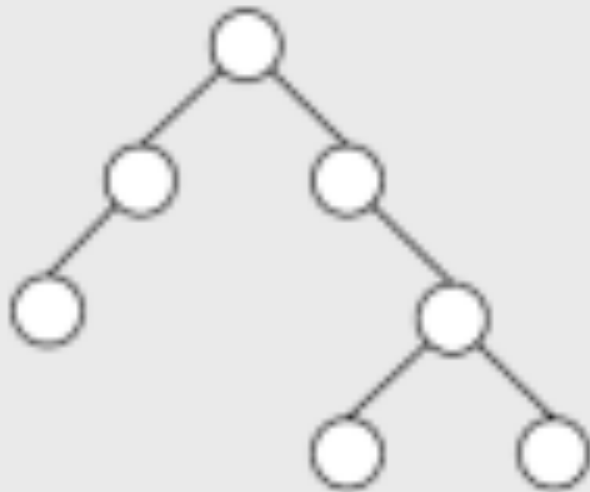


Non-Linear Data Structure

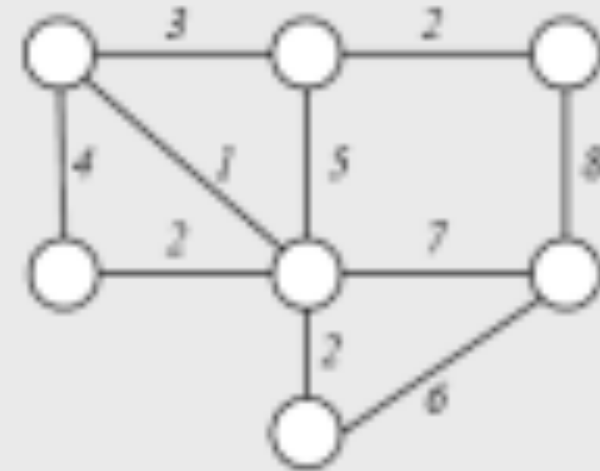
โครงสร้างข้อมูลไม่เชิงเส้น

การจัดเรียงข้อมูลอย่างไม่มีลำดับ
แต่เน้นความสัมพันธ์แบบลำดับชั้นหรือการเชื่อมโยงที่ซับซ้อน

Trees, Graphs



Tree



Graph



Linear Data Structure

โครงสร้างข้อมูลเชิงเส้น

เป็นการจัดเรียงข้อมูลในลำดับ

การเข้าถึงข้อมูลทำได้โดยเริ่มจากจุดแรกไล่ไปที่ละลำดับตามโครงสร้าง

- Arrays
- Linked lists
- Stacks
- Queues



Static Data Structure

โครงสร้างข้อมูลแบบคงที่

โครงสร้างและขนาดถูกระบุตายตัว
ไม่สามารถเปลี่ยนแปลงหลังการสร้าง
เช่น Array

Dynamic Data Structure

โครงสร้างข้อมูลแบบเปลี่ยนแปลงได้

โครงสร้างที่สามารถปรับเปลี่ยนขนาดและ
รูปแบบได้ระหว่างการทำงาน
เช่น Linked Lists



DATA STRUCTURE

Linked-list



Linked-list

เป็นโครงสร้างข้อมูลแบบ **Linear** ชุดข้อมูลที่ถูกจัดเก็บภายในหน่วยที่เรียกว่า **node**
แต่ละ **node** ประกอบด้วยสองส่วนหลัก
คือ **ข้อมูล**ที่เราต้องการเก็บ และ **ส่วนอ้างอิง (ลิงก์)** ที่ชี้ไปยัง **node** ถัดไป

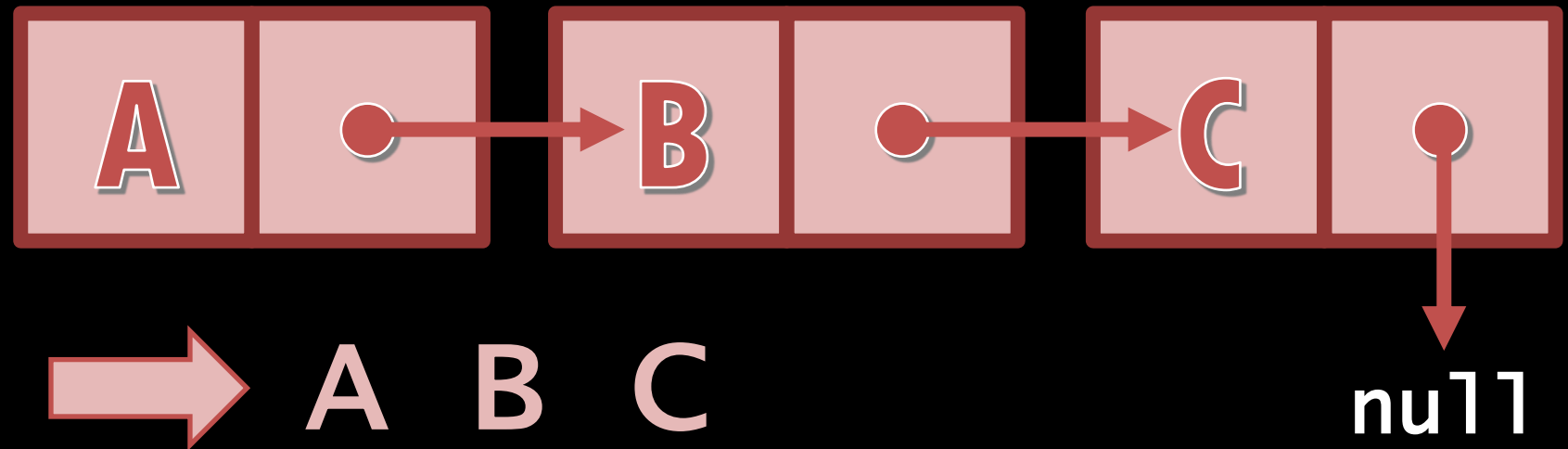




Array

• grade ["A", "B", "C"] << ประกาศเก็บ array 3 ช่อง

Linked-list



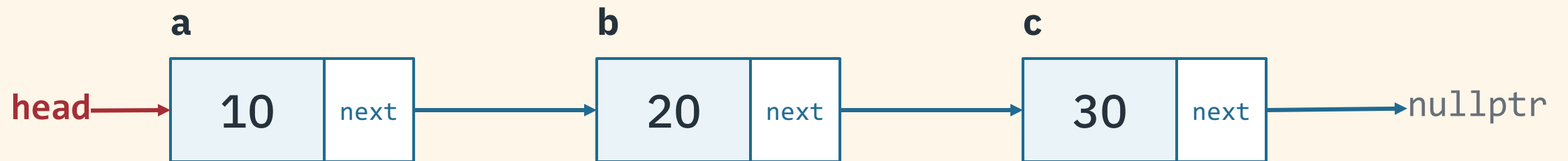
Singly linked-list << สามารถข้อมูลได้ทิศทางเดียว



Linked list เก็บลำดับด้วยการเชื่อมโยง

อาร์เรย์เข้าถึงสมาชิกด้วย index ส่วน linked list เก็บที่อยู่ของโหนดถัดไปไว้ในแต่ละโหนด เราจึงเดินตามการเชื่อมโยงเพื่ออ่านข้อมูลตามลำดับ

ตำแหน่งที่วางโหนดไม่จำเป็นต้องเรียงตามลำดับข้อมูล สิ่งที่กำหนดว่าเราจะไปต่อที่ใดคือค่าของ next





โหนดหนึ่งตัวมีข้อมูลและตัวชี้ไปต่อ

เราสร้าง struct Node ให้มี data สำหรับเก็บข้อมูล และ next สำหรับเก็บที่อยู่ของโหนดถัดไป

ถ้ายังไม่มีโหนดถัดไป เราให้ next เป็น nullptr ซึ่งหมายถึงไม่มีปลายทางที่จะตามต่อในตัวอย่างนี้

```
struct Node {  
    int data;  
    Node* next;  
};  
  
Node a{10, nullptr};
```



จุดและลูกศรใช้เข้าถึงสมาชิกเหมือนกัน

เมื่อมีตัวแปรโหนดโดยตรง เราใช้ a.data แต่ถ้ามี pointer ที่ชี้ไปยังโหนด เราใช้ p->data เพื่อเข้าถึงสมาชิกของโหนดปลายทาง

คำว่า p->data มีความหมายเดียวกับ (*p).data จุดที่ควรระวังคือ p ต้องชี้ไปยังโหนดที่ใช้งานได้ก่อนเสมอ

```
Node a{10, nullptr};  
Node* p = &a;
```

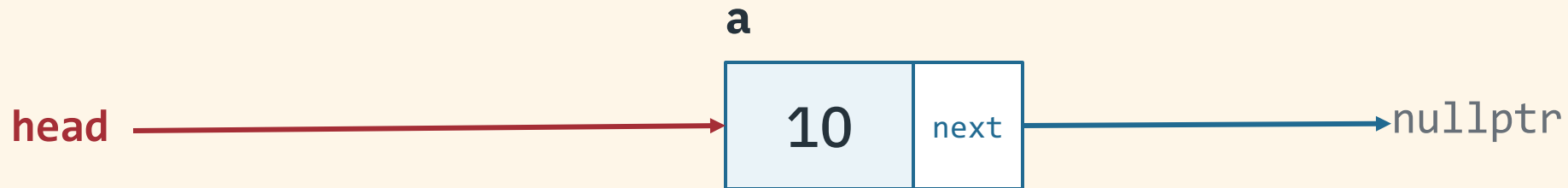
```
std::cout << a.data;  
std::cout << p->data;  
std::cout << (*p).data;
```



Head เก็บจุดเริ่มต้นของรายการ

เราจะใช้ head เก็บที่อยู่ของโหนดแรก เมื่อมีข้อมูลเพียงหนึ่งโหนด head จึงชี้โหนดนั้น และ next ของโหนดนั้นเป็น nullptr

ตัวแปร head เป็น pointer อีกตัวหนึ่ง ไม่ใช่ช่องข้อมูลภายในโหนด

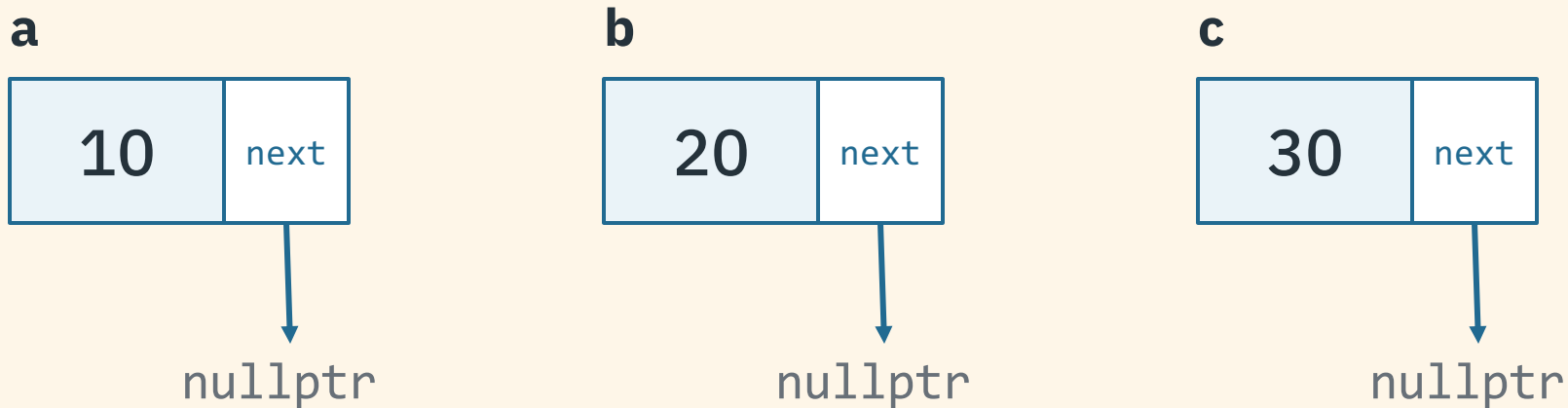




เริ่มจากโหนดที่ยังไม่เชื่อมกัน

ตอนนี้เรามีโหนด a, b และ c แล้ว แต่ next ของแต่ละโหนดยังเป็น nullptr จึงยังเดินจาก a ไปถึง b หรือ c ไม่ได้

ลองทanyaว่าเราต้องเปลี่ยนสมาชิกใด เพื่อให้รายการมีลำดับ 10, 20 และ 30

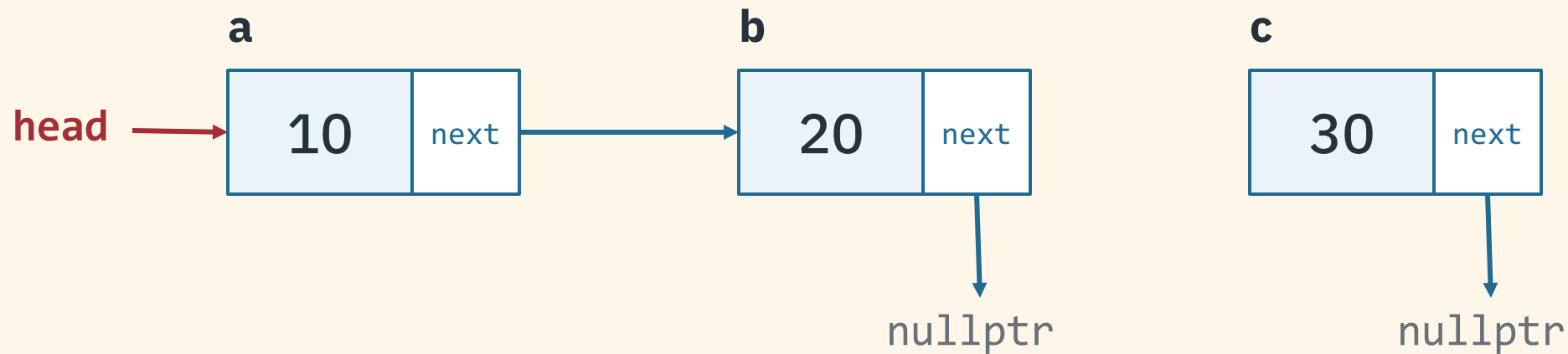




เชื่อมโหนดแรกเข้ากับโหนดที่สอง

คำสั่ง `a.next = &b` ทำให้ช่อง `next` ของโหนด `a` เก็บที่อยู่ของ `b` ดังนั้นเมื่อเริ่มจาก `a` เราจึงตามลิงก์ไปถึง `b` ได้

ในขั้นนี้ `head` ยังชี้ `a` เหมือนเดิม สิ่งที่เปลี่ยนคือสมาชิก `next` ภายในโหนด `a`

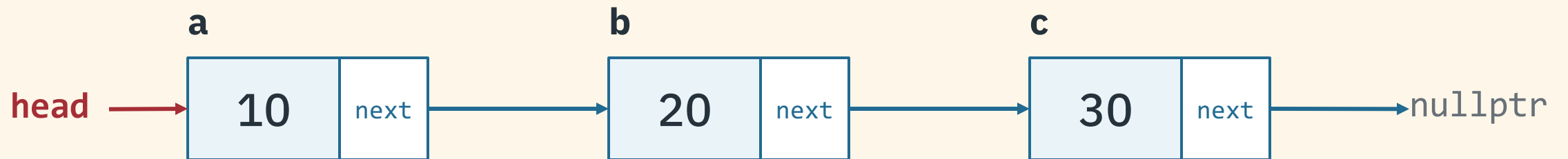




เชื่อมต่อจนได้รายการครบสามโหนด

เมื่อสั่ง `b.next = &c` เราก็เดินจาก `a` ไป `b` แล้วไป `c` ได้ ส่วน `c.next` ยังคงเป็น `nullptr` เพื่อบอกว่ารายการจบที่โหนด `c`

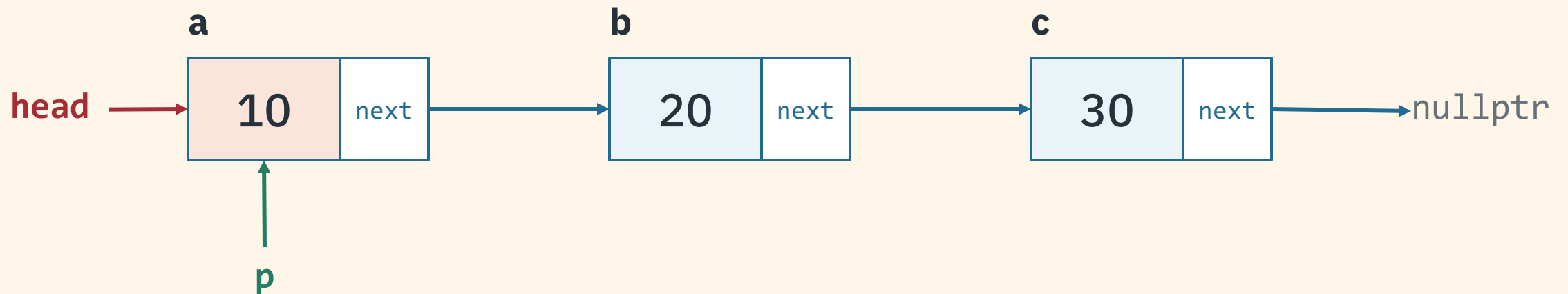
ขอให้อ่านลำดับข้อมูลโดยตามลูกศร แล้วอธิบายว่าทำไมเราจึงรู้ว่าควรหยุดที่ใด





ตัวเดินแรกที่โหนดแรก

เราสร้างตัวเดิน p ให้เริ่มชี้โหนดเดียวกับ $head$ จากนั้นอ่านข้อมูลที่ p ชี้อยู่ แล้วจึงเลื่อนไปยังโหนดถัดไป การมีตัวเดินแยกจาก $head$ ช่วยให้เรายังจำจุดเริ่มต้นของรายการไว้ได้

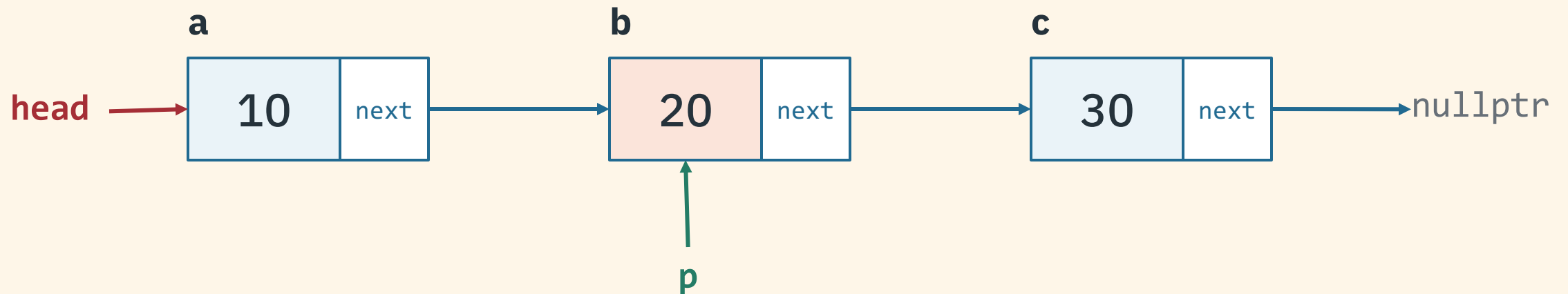




ตัวเดินเปลี่ยนตำแหน่ง แต่ Head ยังอยู่ที่เดิม

หลังสั่ง $p = p \rightarrow \text{next}$ ตัวเดิน p จะชี้โหนด b ซึ่งเก็บค่า 20 แต่ head ยังชี้โหนด a

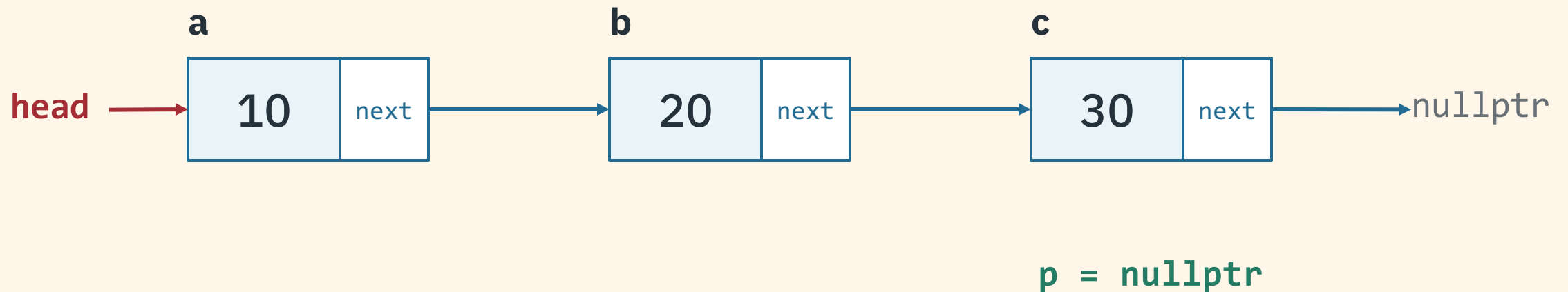
คำสั่งนี้เปลี่ยนค่าของตัวแปร p โดยไม่ได้เปลี่ยนสมาชิก next ของโหนดใด ดังนั้นรูปแบบการเชื่อมโยงเดิมจึงยังอยู่ครบ





การเดินทางลงเมื่อไม่มีโหนดให้ตามต่อ

หลังอ่านโหนด c แล้ว การเลื่อนอีกครั้งจะทำให้ p เป็น nullptr เราจึงหยุดก่อนใช้ p->data อีกครั้ง ลำดับที่อ่านได้คือ 10, 20 และ 30 ส่วน head ยังชี้โหนดแรก จึงเริ่มเดินรายการใหม่ได้





จากภาพการเดินทางรายการสู่คำสั่ง C++

ก่อนอ่านข้อมูล เราตรวจว่า p ไม่เป็น nullptr
เมื่ออ่านเสร็จแล้วจึงเลื่อนไปยัง p->next

ถ้ารายการว่าง head จะเป็น nullptr ตั้งแต่ต้น
ทำให้ loop ไม่ทำงานเลย เราจึงไม่พยายามอ่าน
สมาชิกของโหนดที่ไม่มีอยู่

```
Node* p = head;
while (p != nullptr) {
    std::cout << p->data << " ";
    p = p->next;
}
```



<https://github.com/thunderbirdxs/POSN67.git>

Linked-list

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node *next;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
    }
};

void printList(Node *n) {
    while (n != nullptr) {
        cout << n->data << " ";
        n = n->next;
    }
}
```

```
int main() {
    Node *head = new Node(1);
    Node *second = new Node(2);
    Node *third = new Node(3);

    head->next = second;
    second->next = third;

    printList(head);

    return 0;
}
```

C:\WINDOWS\system32

1 2 3

c:\Users\Peerapat\posn67\01>



<https://github.com/thunderbirdxs/POSN67.git>

Linked-list

```
#include <iostream>
using namespace std;
```

```
class Node {
public:
    int data;
    Node *next;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
    }
};
```

```
void printList(Node *n) {
    while (n != nullptr) {
        cout << n->data << " ";
        n = n->next;
    }
}
```

```
int main() {
    Node *head = new Node(1);
    Node *second = new Node(2);
    Node *third = new Node(3);
```

constructor : node

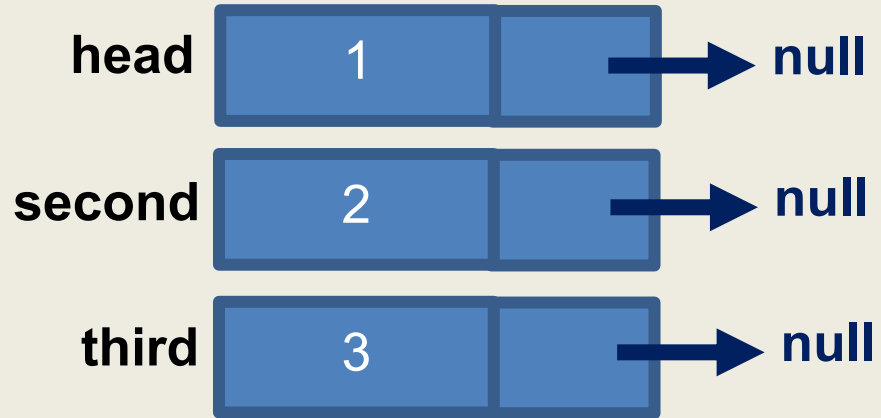


C:\WINDOWS\system32

1 2 3

c:\Users\Peerapat\posn67\01>

Linked-list



```
Node(int data) {  
    this->data = data;  
    this->next = nullptr;  
}  
};  
  
void printList(Node *n) {  
    while (n != nullptr) {  
        cout << n->data << " ";  
        n = n->next;  
    }  
}
```

```
int main() {  
    Node *head = new Node(1);  
    Node *second = new Node(2);  
    Node *third = new Node(3);  
  
    head->next = second;  
    second->next = third;  
  
    printList(head);  
  
    return 0;  
}
```

C:\WINDOWS\system32

1 2 3

c:\Users\Peerapat\posn67\01>



Linked-list

```
#include <iostream>
using namespace std;
```

```
class Node {
```

```
public
```

```
int data;
```

```
Node *next;
```

```
Node(int data) {
```

```
    this->data = data;
```

```
    this->next = nullptr;
```

```
}
```

```
};
```

```
void printList(Node *n) {
```

```
    while (n != nullptr) {
```

```
        cout << n->data << " ";
```

```
        n = n->next;
```

```
    }
```

```
}
```

~~head -> null~~

head -> second

```
int main() {
    Node *head = new Node(1);
    Node *second = new Node(2);
    Node *third = new Node(3);
```

```
    head->next = second;
```

```
    second->next = third;
```

```
    printList(head);
```

```
    return 0;
```

```
}
```



C:\WINDOWS\system32



1 2 3

c:\Users\Peerapat\posn67\01>



Linked-list

```
#include <iostream>
using namespace std;
```

```
class Node {
public:
    int data;
    Node *next;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
    }
};
```

```
void printList(Node *n) {
    while (n != nullptr) {
        cout << n->data << " ";
        n = n->next;
    }
}
```

~~second -> null~~
second -> third

```
int main() {
    Node *head = new Node(1);
    Node *second = new Node(2);
    Node *third = new Node(3);

    head->next = second;
    second->next = third;

    printList(head);

    return 0;
}
```

C:\WINDOWS\system32

1 2 3

c:\Users\Peerapat\posn67\01>



Linked-list

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node *next;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
    }
};

void printList(Node *n) {
    while (n != nullptr) {
        cout << n->data << " ";
        n = n->next;
    }
}
```

```
int main() {
    Node *head = new Node(1);
    Node *second = new Node(2);
    Node *third = new Node(3);

    head->next = second;
    second->next = third;

    printList(head);

    return 0;
}
```

```
C:\WINDOWS\system32\cmd.exe
c:\Users\Peerapat\posn67\01>
```



Linked-list

```
#include <iostream>
using namespace std;
```

```
class Node {
public:
    int data;
    Node *next;

    Node(int data) {
        this->data = data;
        this->next = nullptr;
    }
};
```

```
void printList(Node *n) {
    while (n != nullptr) {
        cout << n->data << " ";
        n = n->next;
    }
}
```

while loop

null -> break

```
int main() {
    Node *head = new Node(1);
    Node *second = new Node(2);
    Node *third = new Node(3);

    head->next = second;
    second->next = third;

    printList(head);

    return 0;
}
```

C:\WINDOWS\system32 x + - □ ×

1 2 3
c:\Users\Peerapat\posn67\01>



กิจกรรม: แทรกโหนดโดยไม่ทำทางเดิมหาย

เราต้องการแทรกโหนด x ซึ่งเก็บค่า 15 ระหว่าง a กับ b โดยยังเก็บโหนด b และ c ไว้ในรายการ

ลองเรียงคำสั่งสองบรรทัดทางขวาให้ถูกต้องลำดับ แล้ววาดลูกศรหลังทำแต่ละบรรทัดก่อนดูคำตอบ

```
a.next = &x;
```

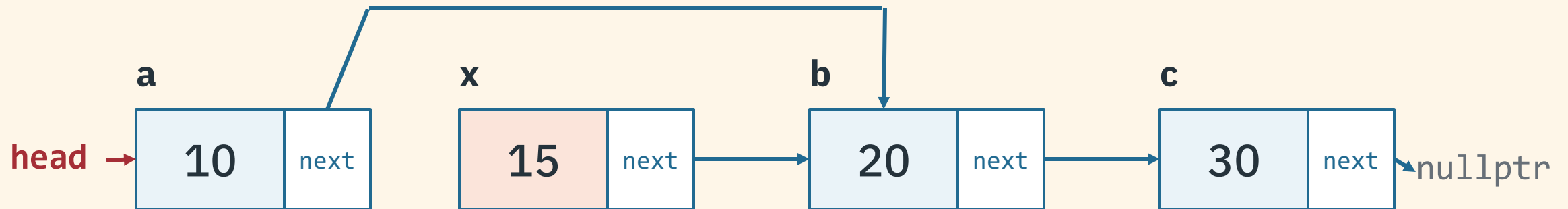
```
x.next = a.next;
```



ขั้นแรก ให้โหนดใหม่จำทางไปต่อก่อน

ก่อนเปลี่ยนลิงก์ของ a เราสั่ง $x.next = a.next$ เพื่อให้ x จำที่อยู่ของ b ไว้ก่อน

ในขั้นนี้รายการเดิมจาก head ยังเป็น a ไป b ไป c แต่โหนด x เตรียมทางไป b ไว้เรียบร้อยแล้ว



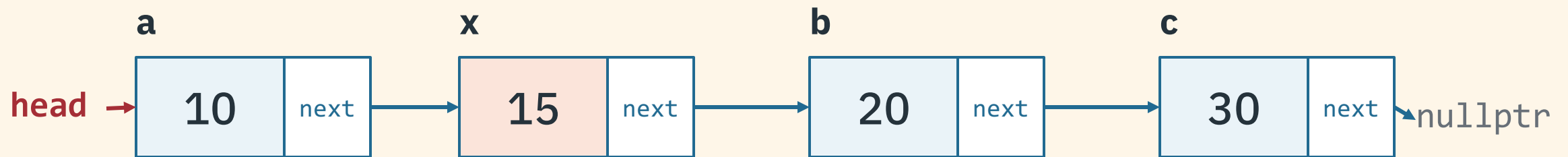
$x.next = a.next;$



ขั้นตอนทำให้โหนดเดิมชี้โหนดใหม่

เมื่อสั่ง $a.next = \&x$ ลำดับจาก head จะเป็น 10, 15, 20 และ 30 เพราะ x เก็บทางไป b ไว้แล้ว

ถ้าสลับสองขั้นตอน $x.next$ จะได้ที่อยู่ของ x เอง ทำให้การเชื่อมโยงผิดและเดินต่อไปส่วนท้ายตามเดิมไม่ได้



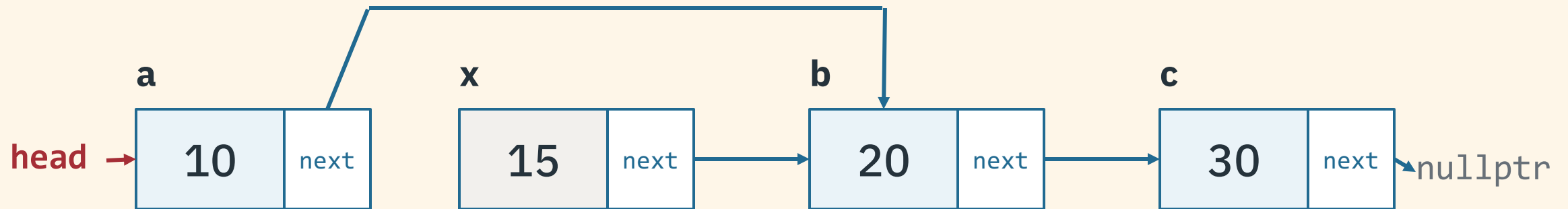
$a.next = \&x;$



การข้ามโหนดเปลี่ยนลิงก์ของรายการ

หากต้องการนำ x ออกจากรายการ เราให้ a.next ชี้ไปยังโหนดที่ x.next ชี้อยู่ รายการจาก head จึงกลายเป็น 10, 20 และ 30

การข้ามโหนดในตัวอย่างนี้ไม่ได้ทำลายตัวแปร x เพียงแต่เราเดินจาก head ไปไม่ถึง x แล้ว



a.next = x.next;



Linked list มีรูปแบบอื่นได้อีก

แบบที่เราใช้ในวันนี้คือ singly linked list ซึ่งแต่ละโหนดรู้จักโหนดถัดไป ส่วน doubly linked list เพิ่มตัวชี้กลับไปยังโหนดก่อนหน้า

Circular linked list เชื่อมโหนดท้ายกลับไปยังจุดเริ่มต้น จึงต้องกำหนดวิธีหยุดให้ต่างจากการรอ nullptr

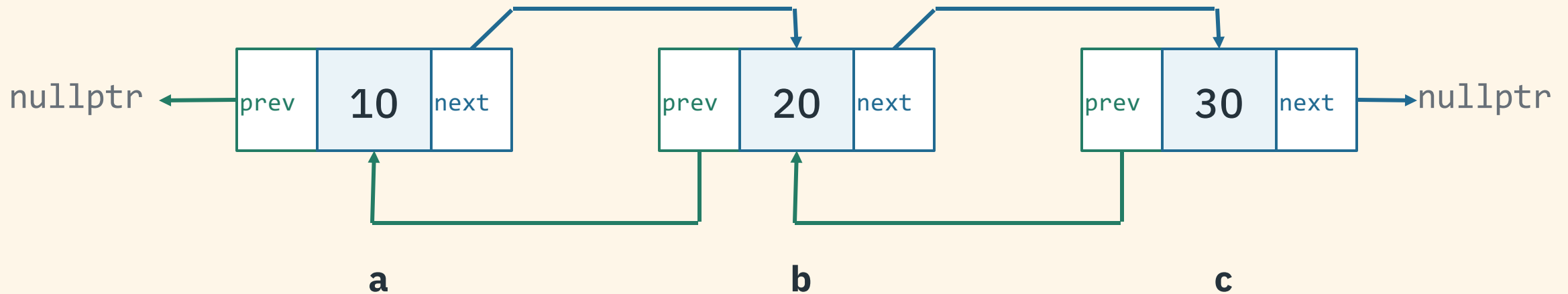
รูปแบบ	สิ่งที่เพิ่มหรือเปลี่ยน
Singly linked list	แต่ละโหนดมี next เพื่อเดินไปข้างหน้า
Doubly linked list	แต่ละโหนดมี prev และ next
Circular linked list	ลิงก์ท้ายกลับเข้าสู่รายการ



Doubly linked list เดินย้อนกลับได้

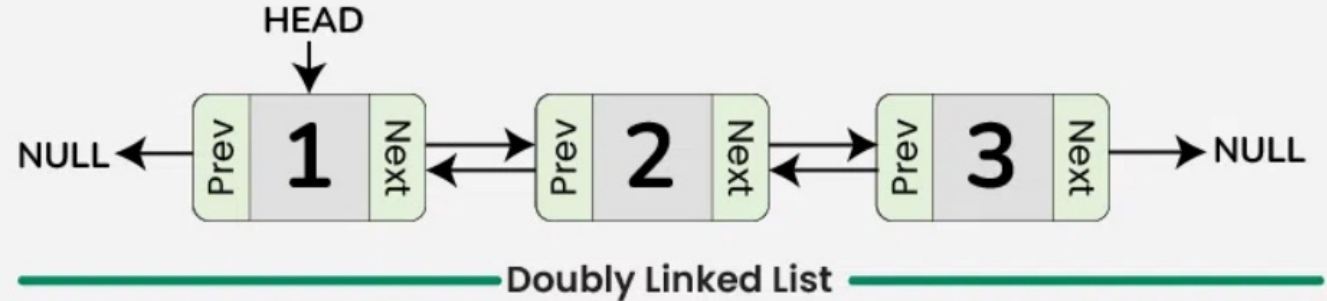
เมื่อแต่ละโหนดมีทั้ง prev และ next เราจึงตามลิงก์ย้อนกลับไปยังโหนดก่อนหน้าได้

อย่างไรก็ตาม เมื่อต้องแทรกหรือนำโหนดออก เราต้องปรับลิงก์ทั้งสองทิศให้สอดคล้องกัน จึงมีรายละเอียดที่ต้องดูแลมากกว่า singly linked list





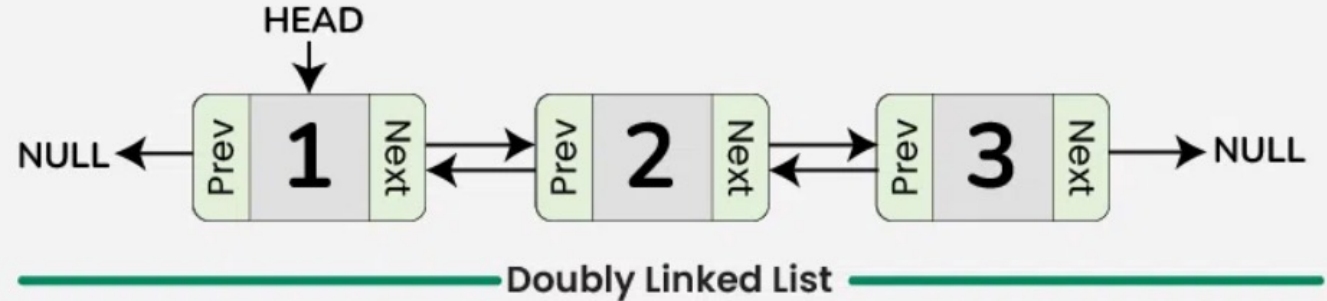
Doubly Linked-list



```
class Node {  
    public:  
    int data;  
    Node *prev;  
    Node *next;  
  
    Node(int data) {  
        this->data = data;  
        this->prev = nullptr;  
        this->next = nullptr;  
    }  
};
```



Doubly Linked-list



```
class Node {
public:
    int data;
    Node *prev;
    Node *next;

    Node(int data) {
        this->data = data;
        this->prev = nullptr;
        this->next = nullptr;
    }
};
```

```
int main() {
    Node *head = new Node(1);
    Node *second = new Node(2);
    Node *third = new Node(3);

    head->next = second;
    second->prev = head;
    second->next = third;
    third->prev = second;

    printList(head);

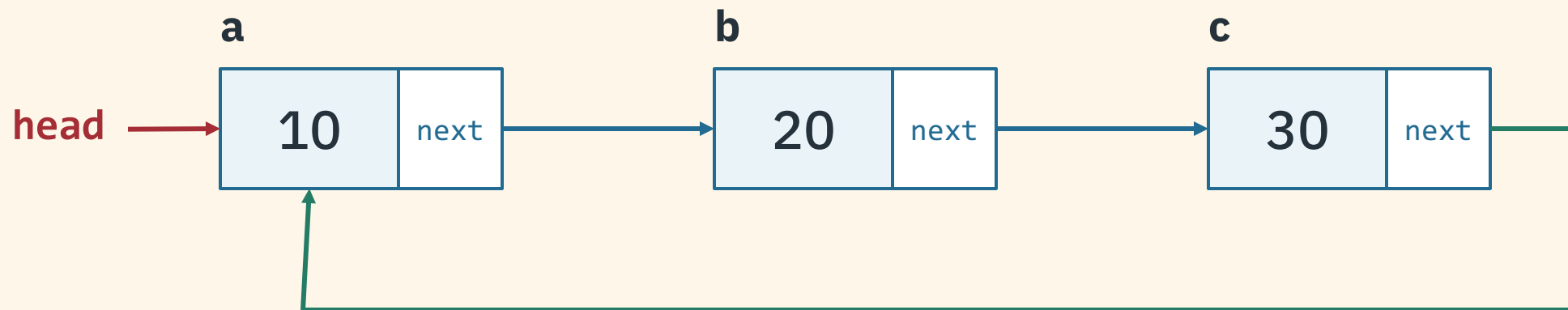
    return 0;
}
```



Circular linked list ต้องมีเงื่อนไขหยุดอีกแบบ

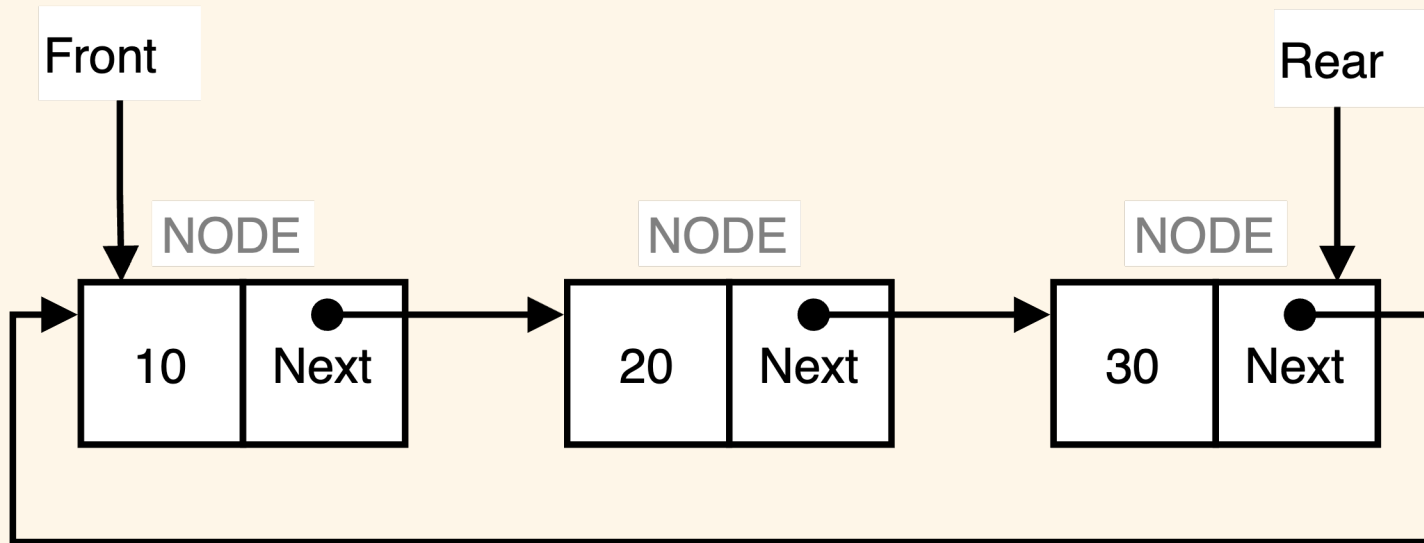
ถ้า next ของโหนดท้ายกลับไปชี้โหนดแรก การเดินจะไม่พบ nullptr ที่ปลายรายการอีกต่อไป

เราจึงต้องกำหนดว่าจะหยุดเมื่อกลับถึงจุดเริ่มต้นหรือเมื่อเดินครบจำนวนที่ต้องการ พร้อมจัดการกรณีรายการว่างก่อนเริ่มเดิน





Circular Linked-list



```
head->next = second;  
second->next = third;  
third->next = head;
```



เปรียบเทียบงานโดยนับสิ่งที่ต้องทำ

หากรู้ตำแหน่งโหนดที่จะแทรกแล้ว การแทรกใน singly linked list เปลี่ยนลิงก์เพียงไม่กี่จุด แต่ถ้ายังไม่รู้ตำแหน่งก็ต้องเดินค้นหาก่อน

ในทางกลับกัน อาร์เรย์เข้าถึงสมาชิกตาม index ได้โดยตรง แต่การแทรกกลางช่วงข้อมูลที่ต้องเรียงติดกันอาจต้องเลื่อนสมาชิกหลายตัว จึงต้องพิจารณางานที่ทำจริง



Linked-list

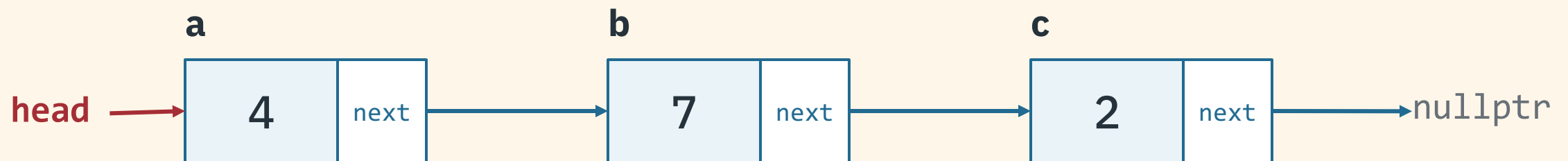




Challenge 2: รวมแຕ้มตามสายโหนด

กำหนด linked list ที่แต่ละโหนดเก็บแຕ้มหนึ่งค่า ให้นักเรียนเดินตาม next แล้วหาผลรวมของแຕ้มทั้งหมด

ให้เขียน `long long sum_list(Node* head)` โดยผู้เรียกเตรียมโหนดให้แล้ว ข้อมูลและลิงก์เดิมต้องคงอยู่ หากรายการว่างให้คืนศูนย์



ผลรวมที่ต้องการคือ 13



ก่อนเริ่มเขียน Challenge 2

รายการไม่มีวงจรมี 0–100000 โหนด แต่ละโหนดมีแอดัม 0–1000000 จึงใช้ long long เก็บผลรวมที่อาจมากกว่าช่วงของ int

ให้ใช้ตัวเดินแยกจาก head และตรวจว่ามีโหนดให้ใช้งานก่อนอ่าน data ทุกครั้ง เริ่มจากการเดินให้ครบก่อนแล้วจึงสะสมผลรวม

ข้อมูลตามลำดับลิงก์	ผลรวม
4 → 7 → 2	13
5	5
รายการว่าง	0



DATA STRUCTURE

Queue - Stack



Queue



Stack



<https://courses.grainger.illinois.edu/cs225/sp2019/notes/quacks/>



Queue

โครงสร้างข้อมูลที่มีการเพิ่มข้อมูลเข้ามาทางด้านท้าย
และการดึงข้อมูลออกทางด้านหน้า

มีประโยชน์ต่อการจัดเรียงลำดับการทำงาน

First In, First Out

FIFO

(เข้าก่อนออกก่อน)





Stack

โครงสร้างข้อมูลแบบ linear ที่มีการ
เพิ่มและดึงข้อมูลออกกับข้อมูลล่าสุดเท่านั้น

Last In, First Out

LIFO

(เข้าหลังออกก่อน)

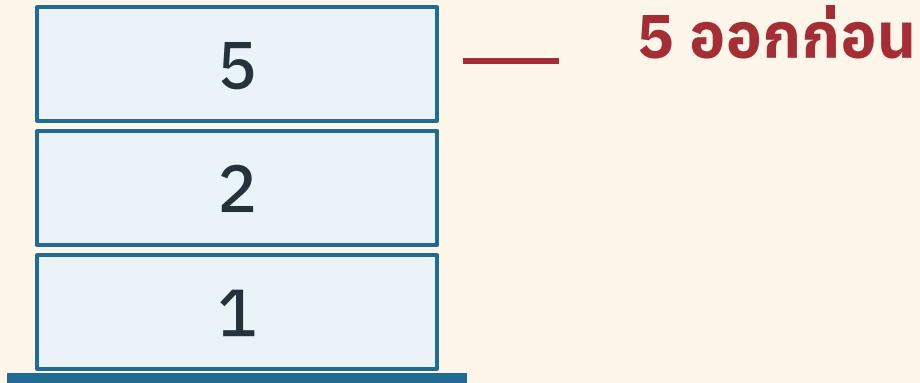


Stack นำข้อมูลล่าสุดออกก่อน

Stack เป็นโครงสร้างที่เพิ่มข้อมูลและนำข้อมูลออกทางด้านบนสุด เมื่อใส่ 1 แล้วใส่ 2 ตามด้วย 5 ข้อมูลที่ออกก่อนจึงเป็น 5

กติกานี้เรียกว่า Last In, First Out หรือ LIFO ซึ่งเหมาะกับงานที่ต้องย้อนกลับจากสิ่งที่เกิดล่าสุด

ใส่ $1 \rightarrow 2 \rightarrow 5$

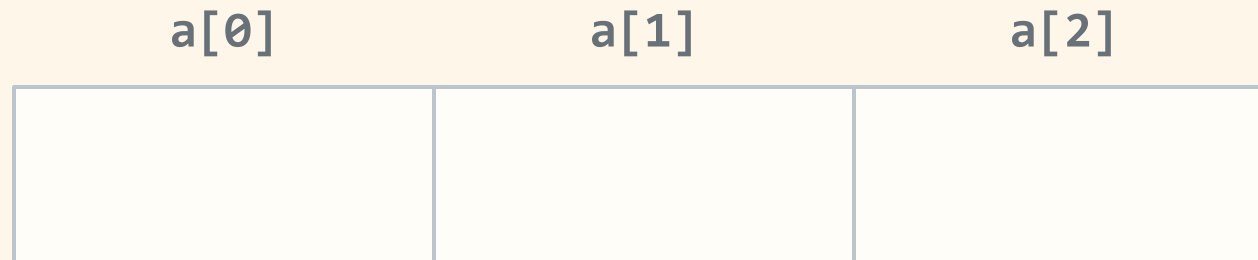




Stack ที่ว่างยังไม่มีสมาชิกบนสุด

เราจะจำลอง stack ด้วยอาร์เรย์ และใช้ top เก็บ index ของสมาชิกบนสุด เมื่อยังไม่มีข้อมูลให้ top เป็น -1

ในสถานะนี้ยังอ่าน $a[top]$ ไม่ได้ เพราะ -1 ไม่ใช่ index ที่ใช้งานได้ของอาร์เรย์



$top = -1$

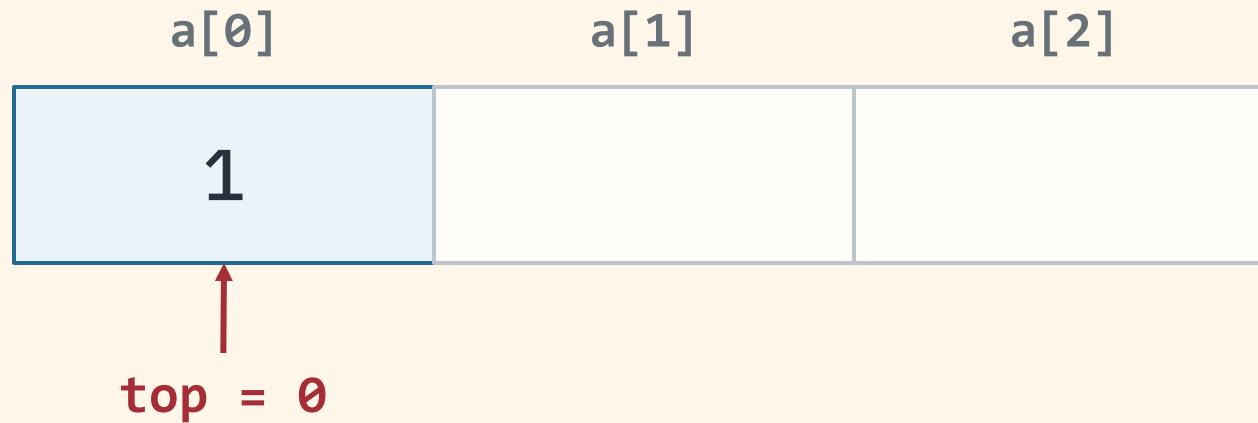
ยังไม่มีสมาชิกให้อ่าน



เมื่อ Push ค่าแรก ข้อมูลจะอยู่ที่ตำแหน่งศูนย์

เมื่อเพิ่มค่า 1 เข้า stack เราเลื่อน top จาก -1 เป็น 0 แล้วเก็บข้อมูลไว้ใน a[0]

ตอนนี้ stack มีสมาชิกหนึ่งตัว และสมาชิกบนสุดมีค่าเป็น 1



สมาชิกบนสุดมีค่า 1



เมื่อเพิ่มข้อมูลอีก สมาชิกบนสุดจะเปลี่ยน

หลังเพิ่ม 2 และ 5 ตามลำดับ top จะมีค่าเป็น 2 และข้อมูลบนสุดคือ $a[2]$ ซึ่งเก็บค่า 5

ค่าที่เข้ามาก่อนยังอยู่ใน stack เพียงแต่เราต้องนำข้อมูลที่อยู่ด้านบนออกก่อนจึงจะเข้าถึงค่าด้านล่างตามกติกา

$a[0]$	$a[1]$	$a[2]$
1	2	5

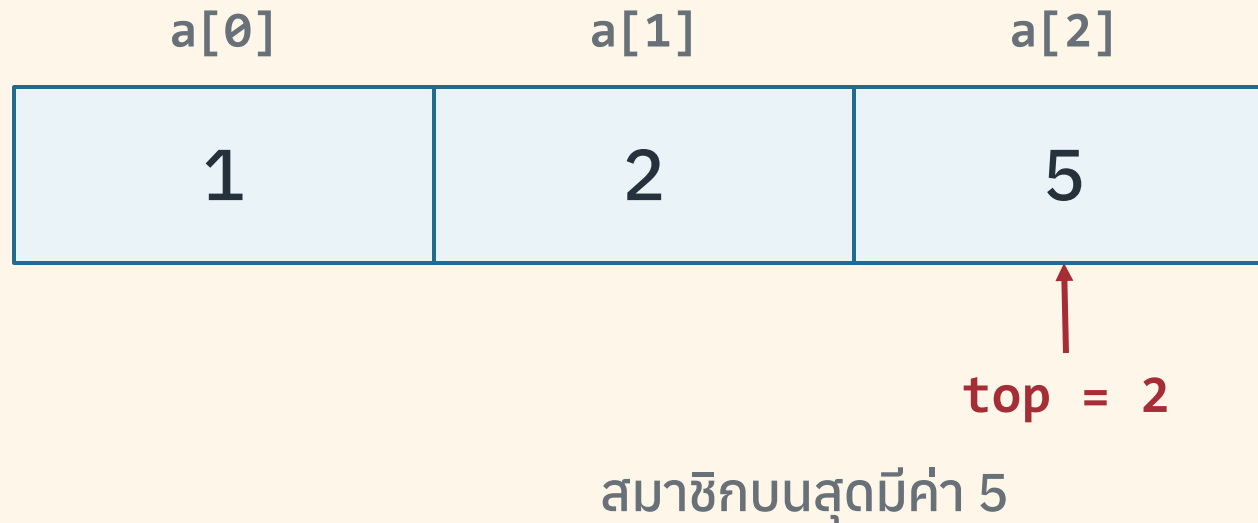
$\uparrow$
 $top = 2$

สมาชิกบนสุดมีค่า 5



Top ใช้ดูข้อมูล โดยยังไม่นำออก

การดูสมาชิกบนสุดทำให้เรารู้ว่าค่านั้นคือ 5 แต่ค่าของตัวแปร top และจำนวนสมาชิกยังคงเดิม
ดังนั้น หากดูบนสุดสองครั้งติดกันโดยไม่มีการเปลี่ยน stack เราจะได้ค่า 5 ทั้งสองครั้ง

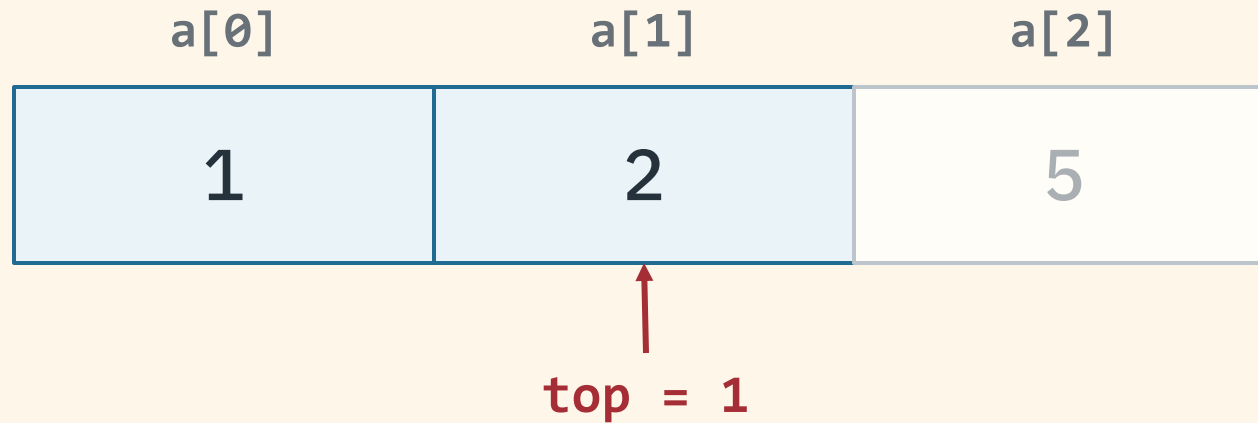




Pop นำสมาชิกบนสุดออกจาก Stack

เมื่อนำค่า 5 ออก เราลด top จาก 2 เป็น 1 สมาชิกบนสุดตัวใหม่จึงเป็นค่า 2

ช่องอาร์เรย์เดิมอาจยังมีเลข 5 อยู่ แต่ช่องนั้นไม่อยู่ในช่วงสมาชิกของ stack แล้ว เราไม่จำเป็นต้องล้างทุกช่องให้เป็นศูนย์



ช่องสีจางไม่เป็นสมาชิกของ stack แล้ว



ก่อนเพิ่มหรือนำออก ต้องตรวจสอบสถานะ

ถ้าอาร์เรย์ได้เพียงสามค่า เมื่อ stack มีสามค่าแล้ว เราต้องจัดการกรณีเต็มก่อนเพิ่มข้อมูลอีกค่า
ในทำนองเดียวกัน เมื่อ stack ว่าง เราจะดูบนสุดหรือนำสมาชิกออกไม่ได้ การเขียนเลขขอบเขตอาร์เรย์
ไม่ใช่การจัดการสถานะเต็ม

ว่าง

$\text{top} = -1$

เต็ม

$\text{top} = \text{capacity} - 1$

ต้องตรวจสอบสถานะก่อนอ่าน เพิ่ม หรือนำข้อมูลออก



คำสั่งสั้น ๆ ที่อยู่เบื้องหลัง Stack

เมื่อแน่ใจว่ายังมีที่ว่าง เราเพิ่ม top แล้วเก็บค่าที่ตำแหน่งใหม่ ส่วนการนำออกทำได้ด้วยการลด top หลังตรวจสอบว่าไม่ว่าง

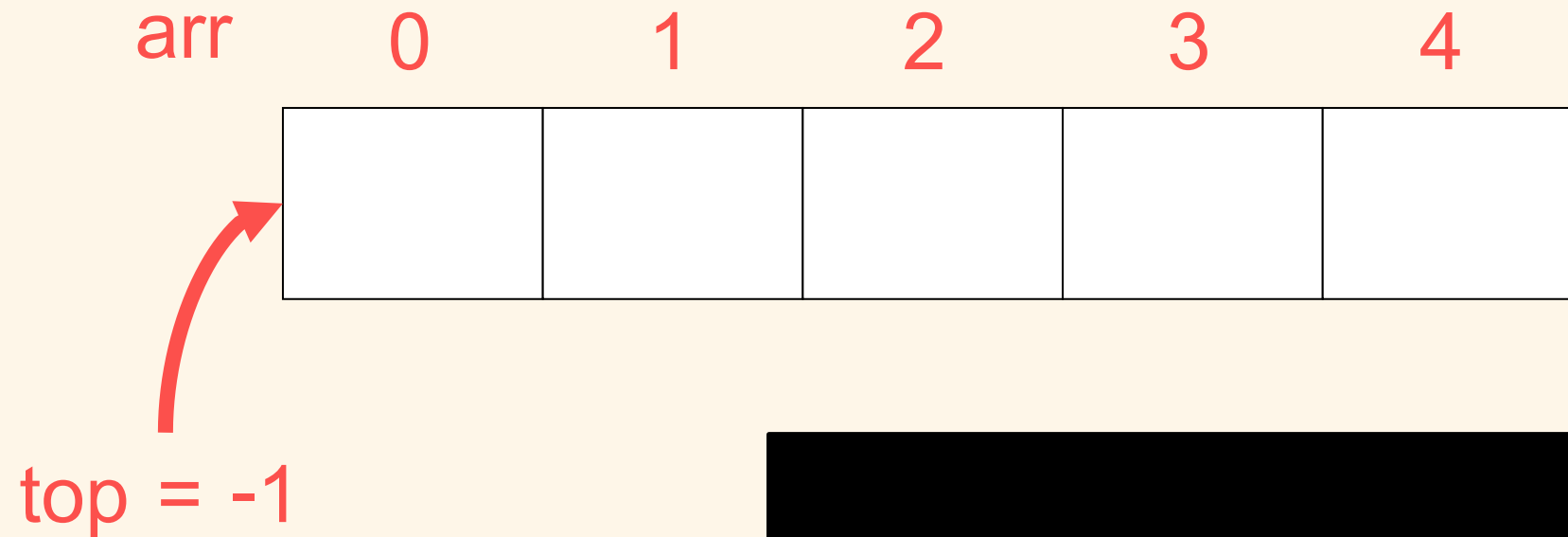
ลองเติมเงื่อนไขในช่องว่างให้ครบ ก่อนเชื่อมแนวคิดนี้กับ `std::stack` ที่เตรียมการทำงานเหล่านี้ไว้ให้แล้ว

```
// Before push
if ( /* still has space */ ) {
    a[++top] = value;
}

// Before pop
if ( /* is not empty */ ) {
    --top;
}
```



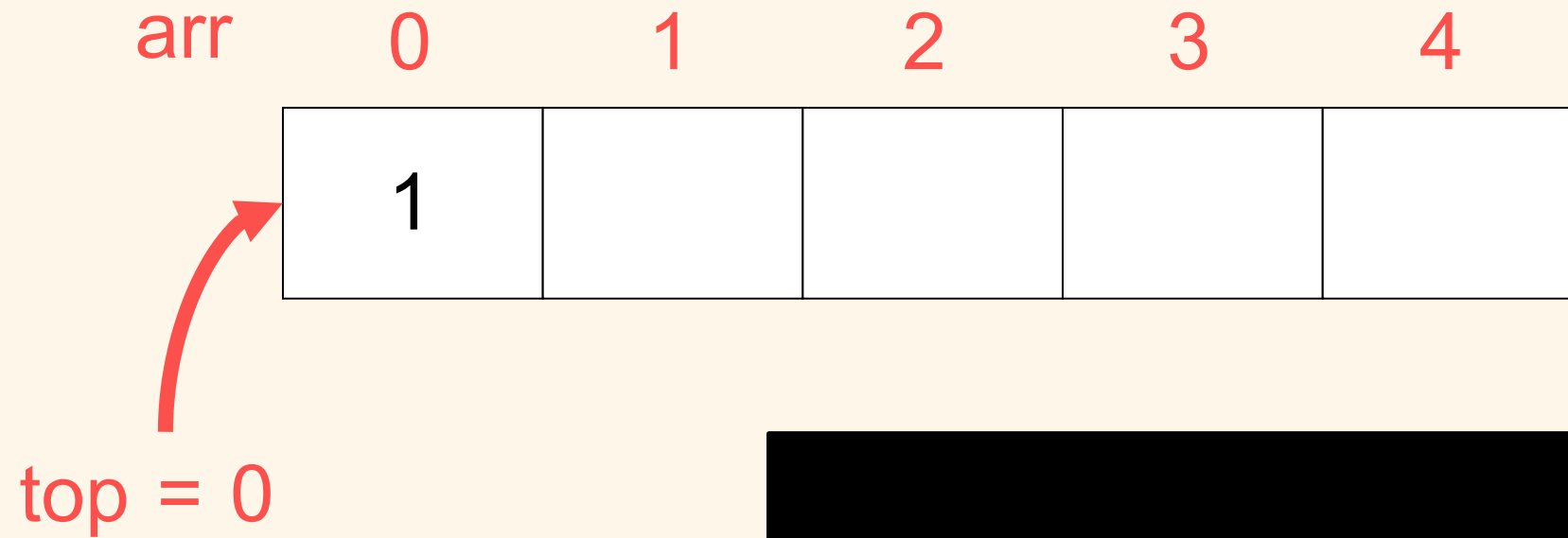
Array-based Stack



`arr[top]`



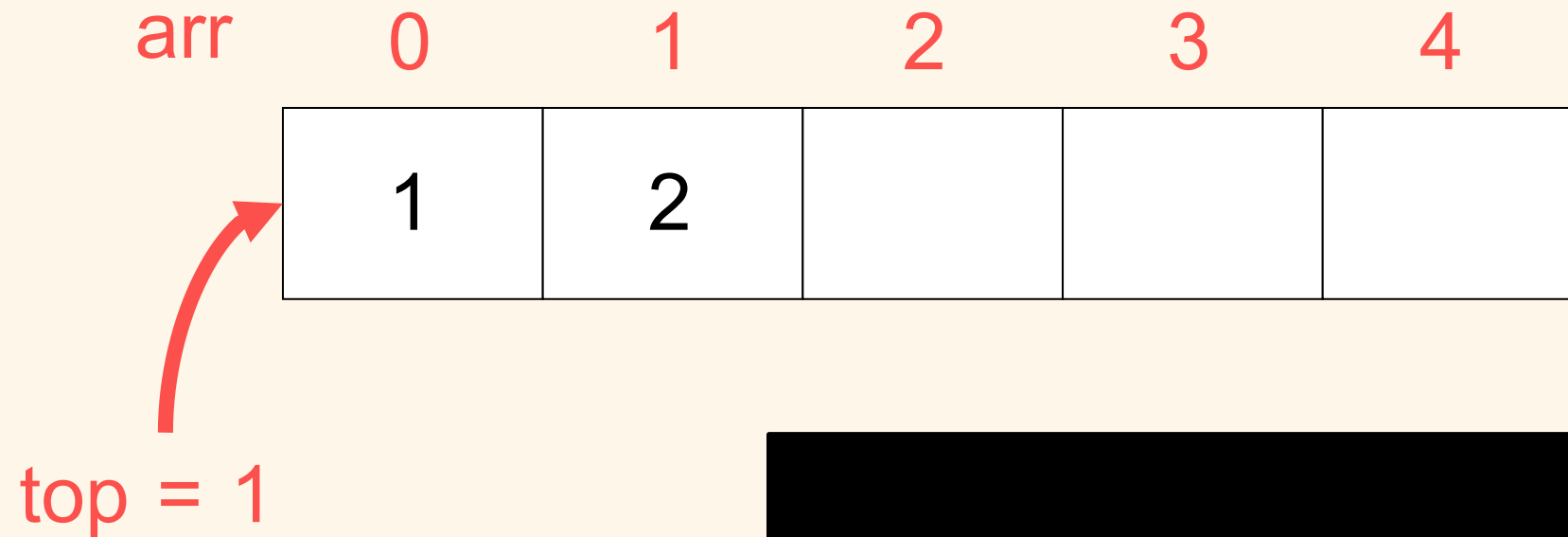
Array-based Stack



`arr[top]`



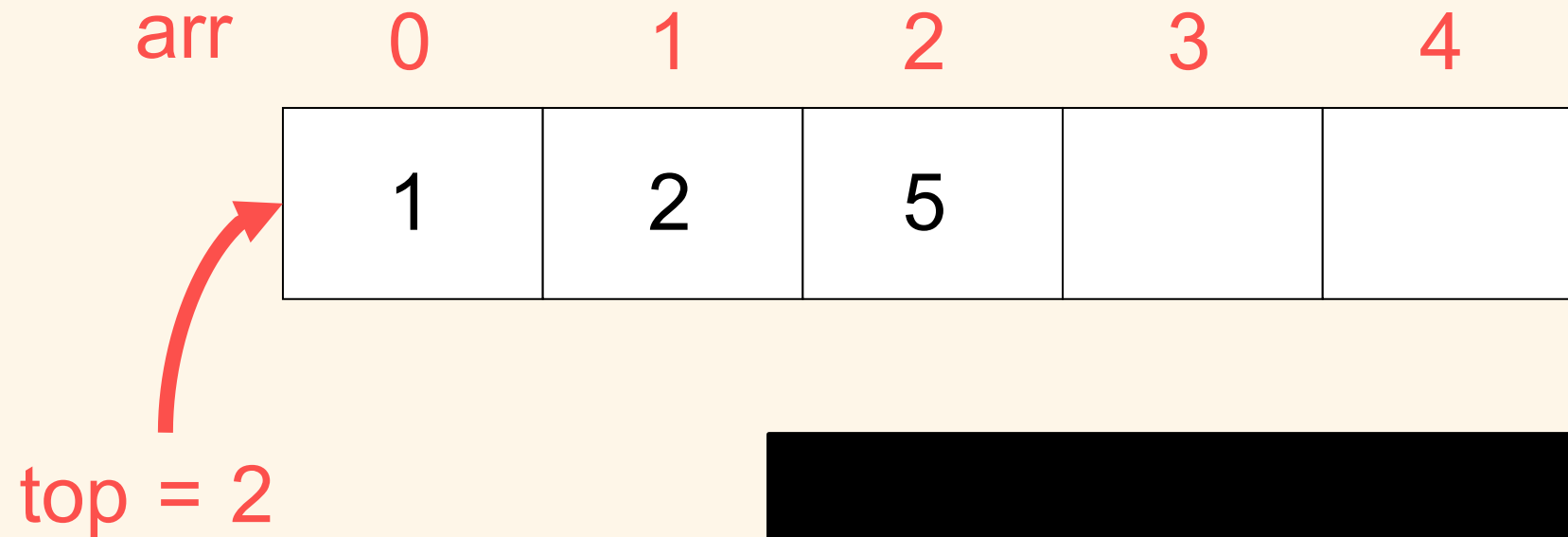
Array-based Stack



`arr[top]`



Array-based Stack



`arr[top]`



Array-based Stack

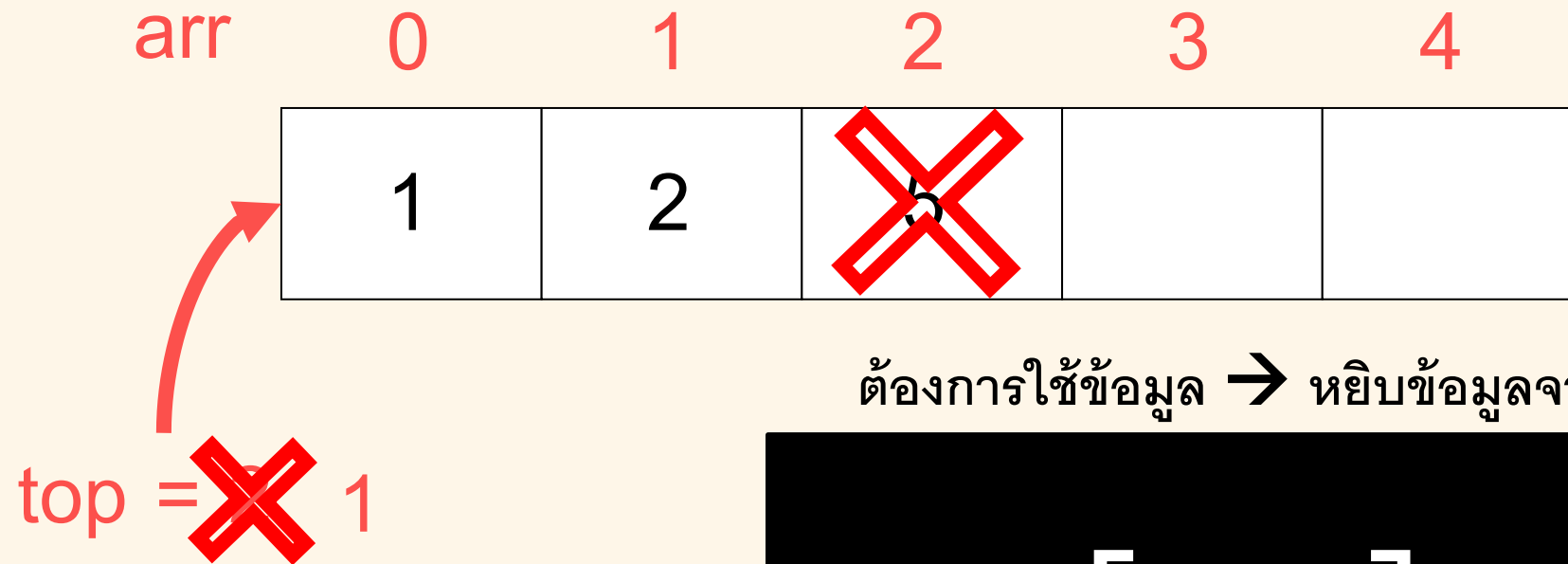


ต้องการใช้ข้อมูล → หยิบข้อมูลจาก index ตัวล่าสุด

`arr[top] → 5`



Array-based Stack

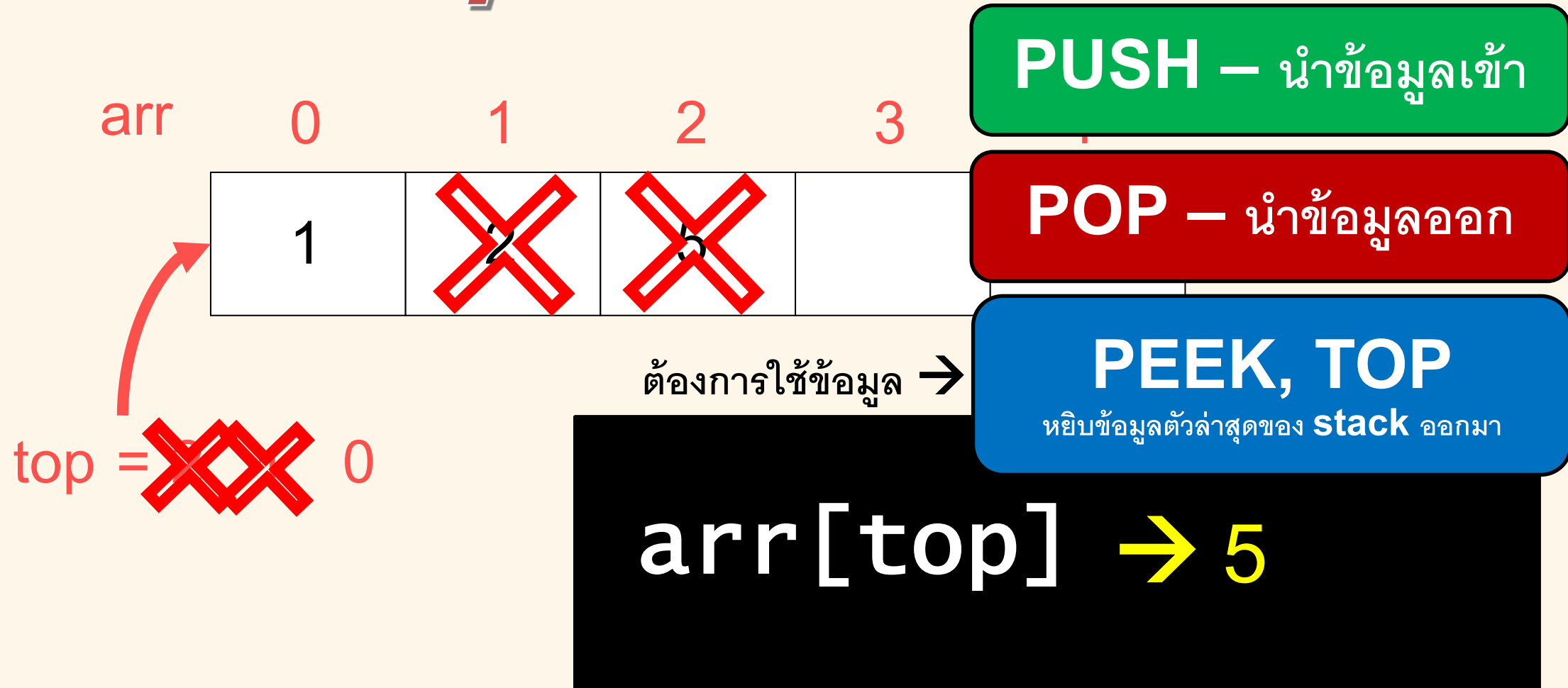


ต้องการใช้ข้อมูล → หยิบข้อมูลจาก index ตัวล่าสุด

arr[top] → 5



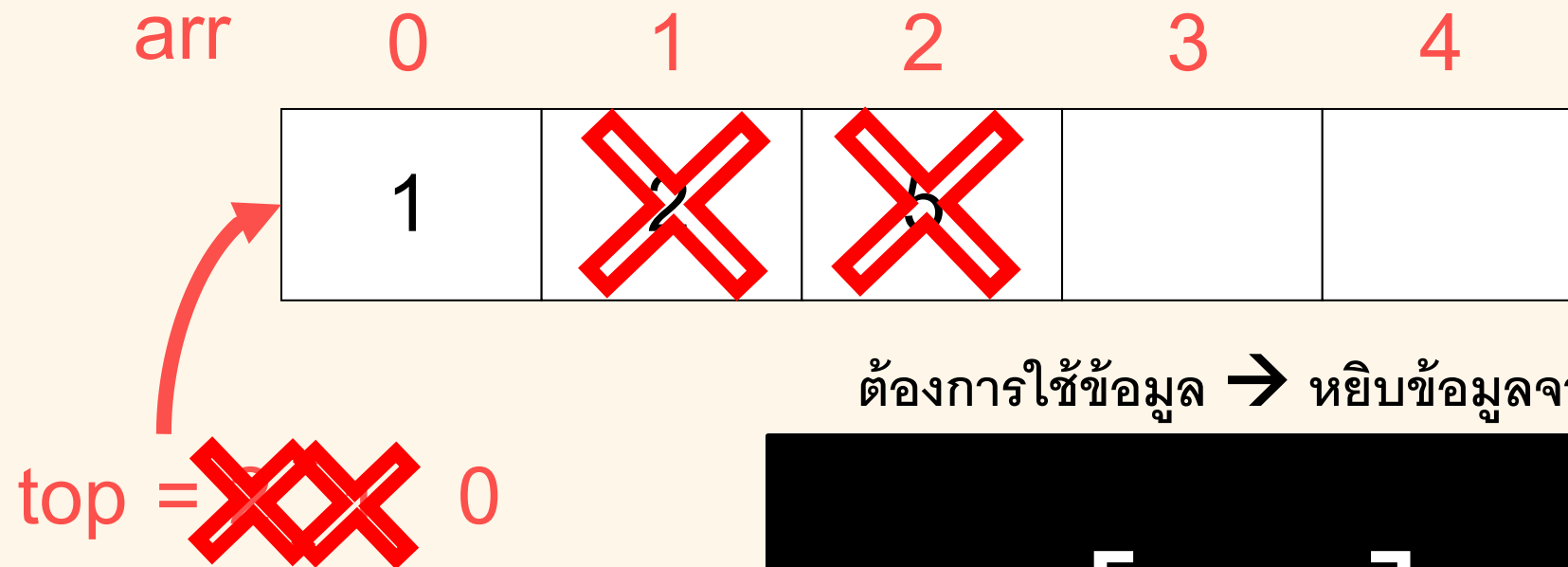
Array-based Stack





Array-based St

PUSH ข้อมูลเกิน array
= Stack Overflow



ต้องการใช้ข้อมูล → หยิบข้อมูลจาก index ตัวล่าสุด

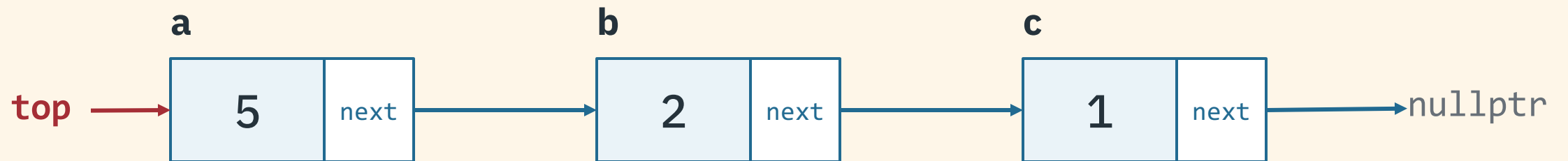
arr[top] → 5



Stack สร้างด้วย Linked list ได้เช่นกัน

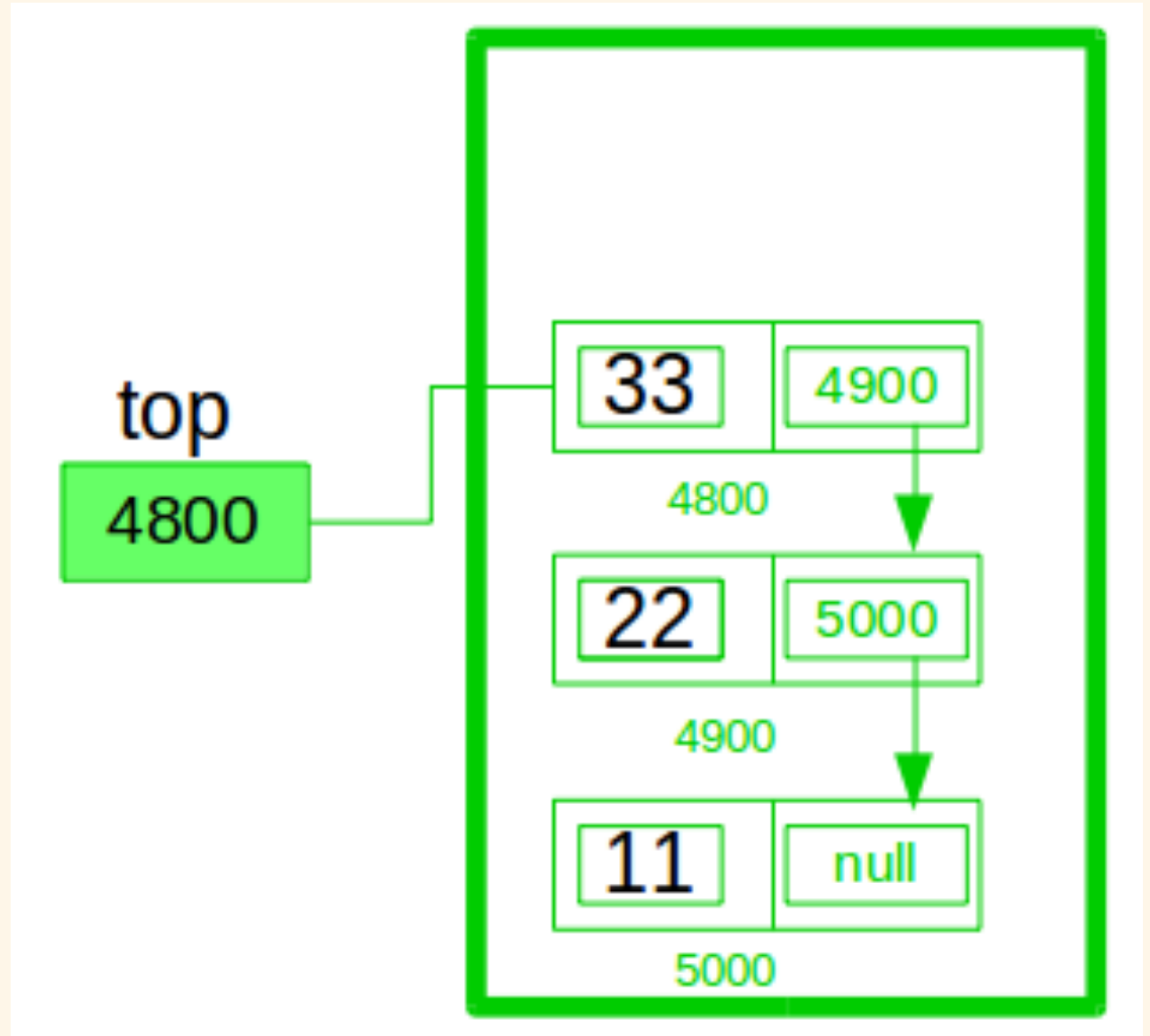
เราสามารถให้โหนดแรกทำหน้าที่เป็นสมาชิกบนสุดของ stack การเพิ่มข้อมูลจึงเป็นการแทรกโหนดใหม่ที่หัวรายการ

การนำข้อมูลออกคือให้ตัวชี้บนสุดเลื่อนไปยังโหนดถัดไป ทิศทาง LIFO ยังคงเดิม แม้วิธีเก็บข้อมูลจะต่างจากแบบอาร์เรย์



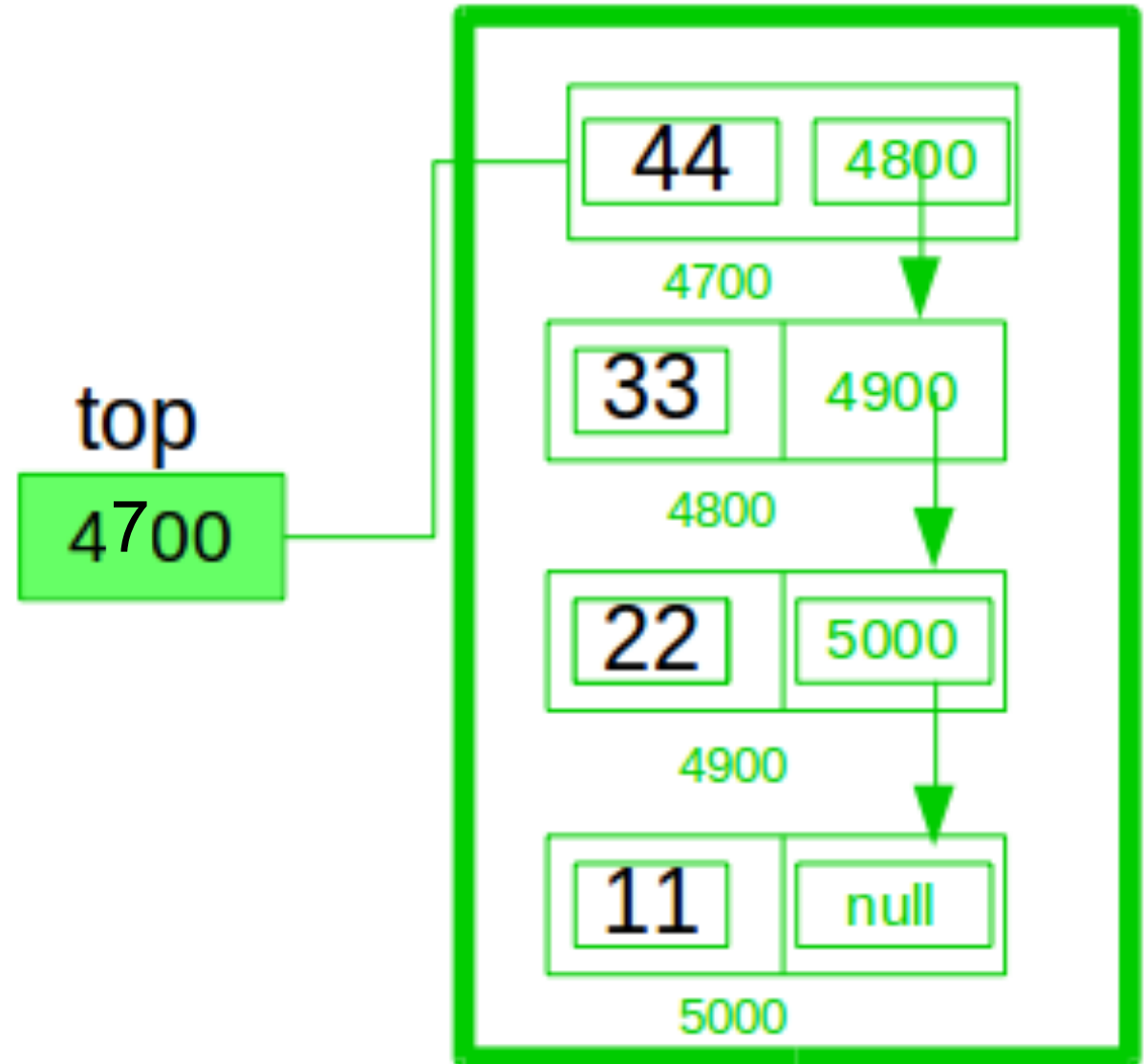


Linked list based Stack



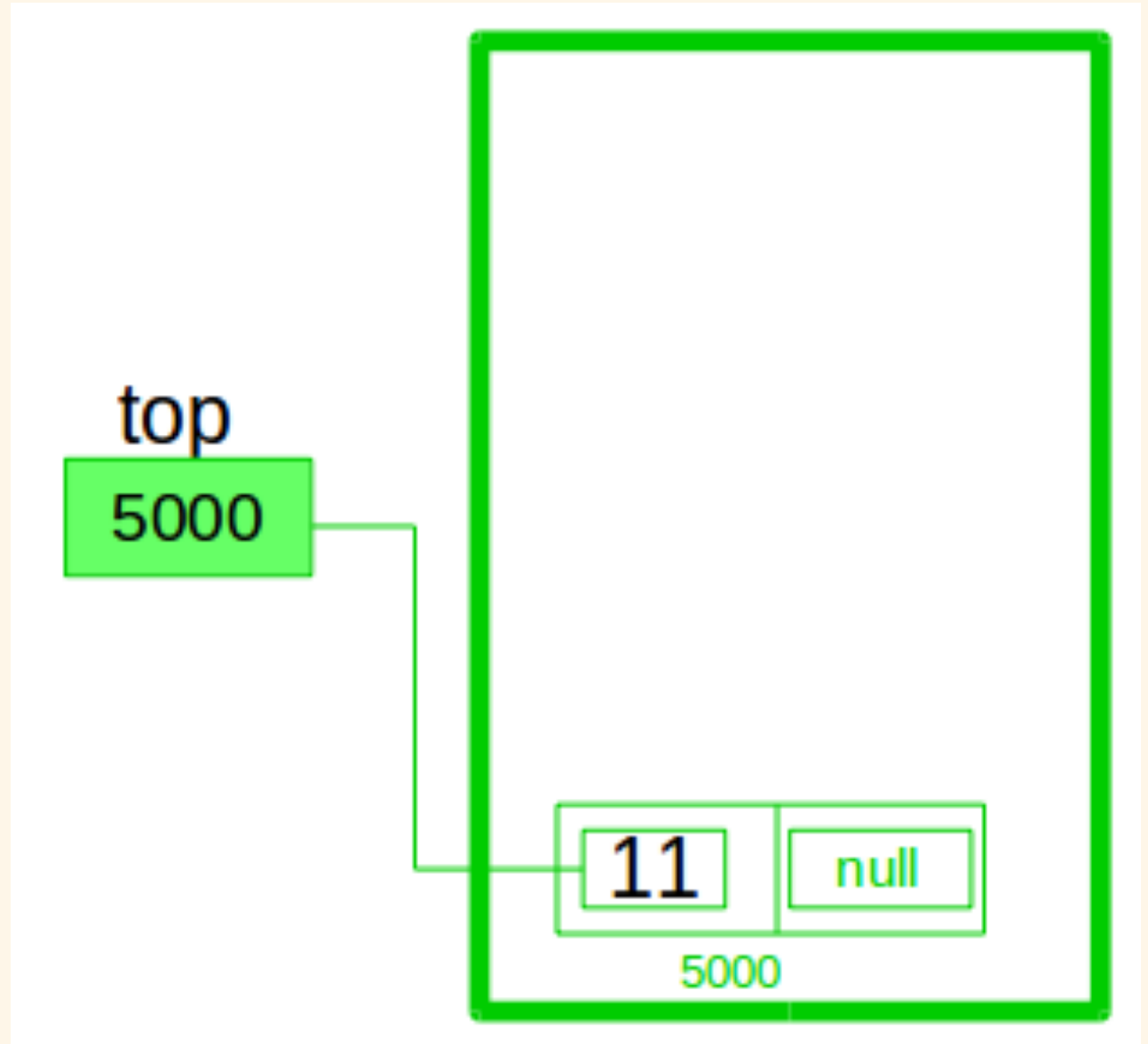


Linked list based Stack

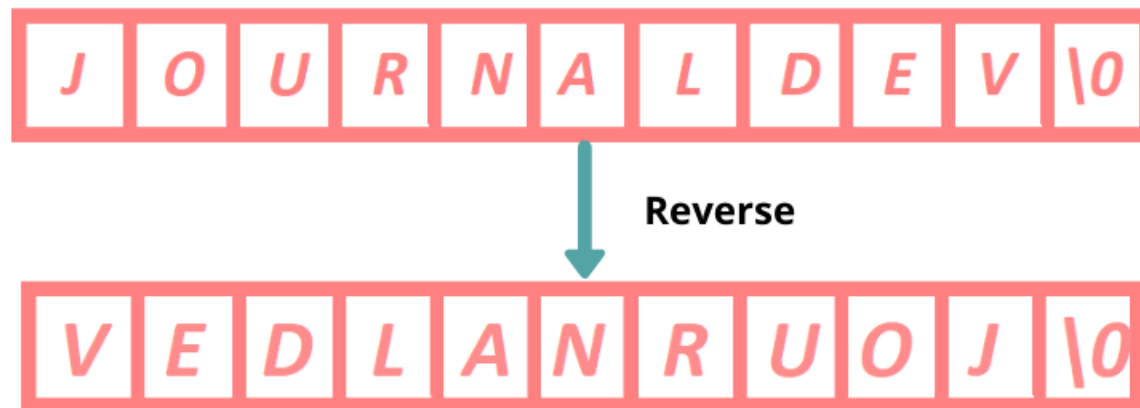
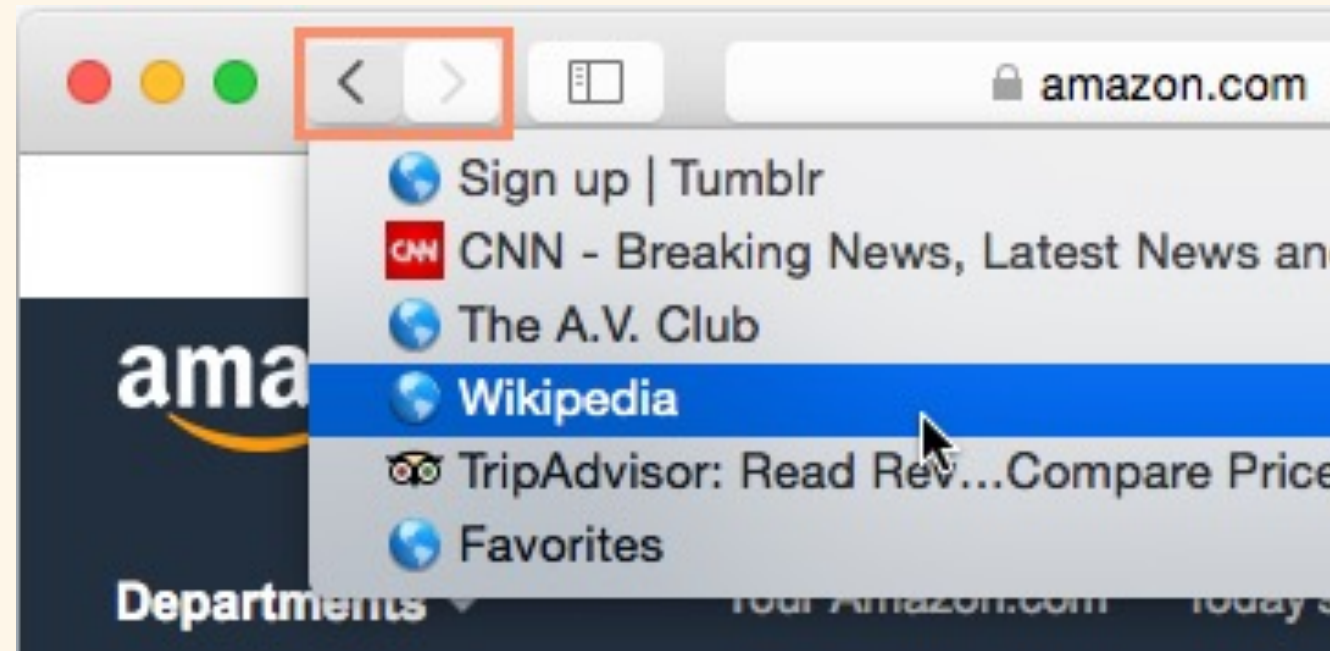
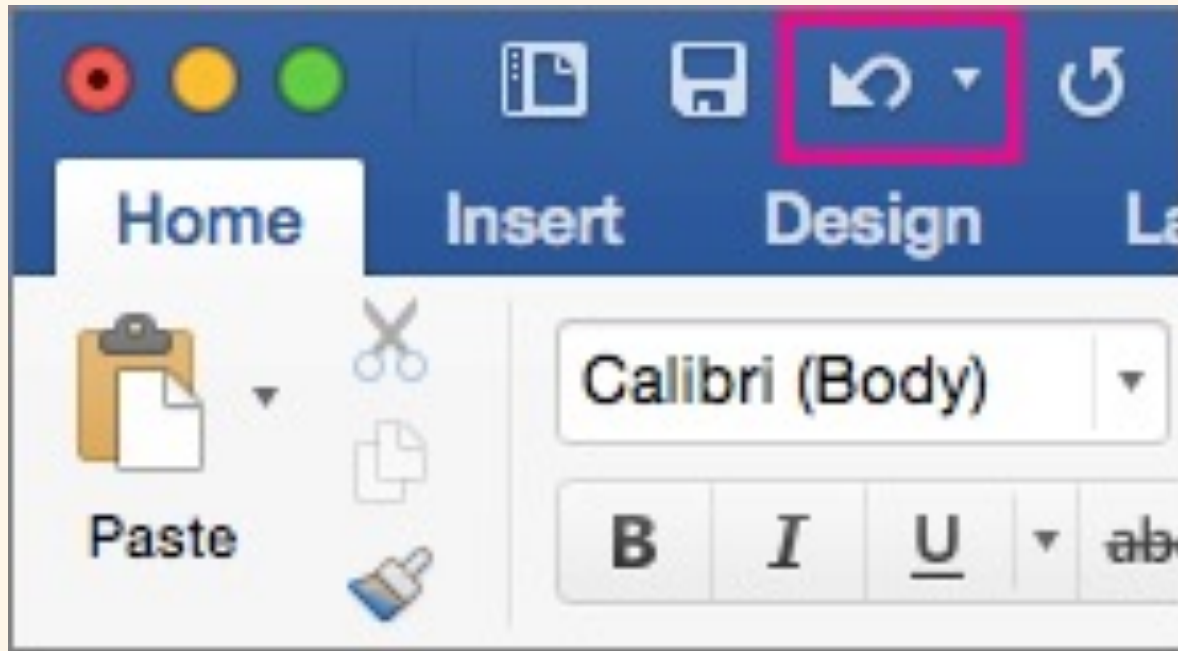




Linked list based Stack



สามารถเจอ Stack ได้จากไหน?





std::stack เตรียมการทำงานของ Stack ไว้ให้

เราประกาศ `std::stack<int>` เพื่อเก็บจำนวนเต็ม และใช้คำสั่งตามกติกา LIFO โดยไม่ต้องดูแล index ของอาร์เรย์ด้วยตนเอง

จุดสำคัญคือ `top()` ใช้เข้าถึงสมาชิกบนสุด ส่วน `pop()` ให้นำสมาชิกออกและไม่คืนค่าข้อมูล จึงต้องอ่านค่าก่อนหากเรายังต้องใช้มัน

คำสั่ง	ความหมาย
<code>push(x)</code>	เพิ่ม x เป็นสมาชิกบนสุด
<code>top()</code>	เข้าถึงสมาชิกบนสุดโดยยังไม่นำออก
<code>pop()</code>	นำสมาชิกบนสุดออก โดยไม่คืนค่า
<code>empty()</code> และ <code>size()</code>	ตรวจสอบว่างและดูจำนวนสมาชิก



อ่านค่าให้เรียบร้อยก่อน Pop

ในตัวอย่างนี้ value รับสำเนาของสมาชิกบนสุด
ก่อนที่เราจะนำสมาชิกนั้นออกจาก stack หลัง
pop แล้ว value จึงยังเป็น 20

เงื่อนไข `!s.empty()` ทำให้เราทำงานกับสมาชิก
บนสุดเฉพาะตอนที่ stack มีข้อมูลอยู่

```
#include <stack>

std::stack<int> s;
s.push(10);
s.push(20);
if (!s.empty()) {
    int value = s.top();
    s.pop();
}
```



Challenge 3: กระดานคะแนน Undo

เริ่มต้นกระดานคะแนนที่ศูนย์ คำสั่ง ADD x เพิ่มคะแนน คำสั่ง UNDO ย้อนการเพิ่มคะแนนล่าสุดที่ยังไม่ถูกย้อน และ SHOW แสดงคะแนนปัจจุบัน

ถ้าไม่มีประวัติให้ย้อน คำสั่ง UNDO ต้องคงคะแนนเดิม ให้นักเรียนใช้ stack เพื่อจำข้อมูลที่จำเป็นต่อการย้อนการทำงาน

0

เริ่มต้น

10

หลัง ADD 10

15

หลัง ADD 5

เมื่อ UNDO หนึ่งครั้ง คะแนนกลับเป็น 10



ตัวอย่างการย้อนคะแนน

รับจำนวนคำสั่งไม่เกิน 100000 แล้วอ่านคำสั่งทีละรายการ ค่า x ของ ADD อยู่ในช่วง 0–1000000 ให้ใช้ long long เก็บคะแนน และให้ SHOW แสดงคะแนนบรรทัดละค่า

หลัง ADD 10 และ ADD 5 คะแนนเป็น 15 เมื่อ UNDO หนึ่งครั้ง คะแนนกลับเป็น 10 ลองคิดว่าควรเก็บข้อมูลอะไรไว้ก่อนเพิ่มคะแนน

คำสั่ง	คะแนนหลังคำสั่ง
ADD 10	10
ADD 5	15
UNDO	10
SHOW	แสดง 10



Queue

โครงสร้างข้อมูลที่มีการเพิ่มข้อมูลเข้ามาทางด้านท้าย
และการดึงข้อมูลออกทางด้านหน้า

มีประโยชน์ต่อการจัดเรียงลำดับการทำงาน

First In, First Out

FIFO

(เข้าก่อนออกก่อน)





ลองทาลำดับอีกครั้ง

เมื่อใส่ 10, 20 และ 30 ตามลำดับ ถ้าใช้ stack แล้วนำออกสองครั้ง เราจะได้ค่าใดบ้าง

หากเปลี่ยนกติกาให้ข้อมูลที่เข้ามาก่อนออกก่อน คำตอบจะต่างออกไปอย่างไร ขอให้แต่ละคู่อธิบายเหตุผลก่อนดูตัวอย่างต่อไป

10

เข้ามาลำดับที่ 1

20

เข้ามาลำดับที่ 2

30

เข้ามาลำดับที่ 3



Queue ให้ข้อมูลที่รอมาก่อนออกก่อน

Queue เพิ่มข้อมูลทางท้ายและนำข้อมูลออกทางหัว จึงรักษาลำดับว่าใครเข้ามาก่อนคนนั้นออกก่อน
กติกานี้เรียกว่า First In, First Out หรือ FIFO เช่น แบบจำลองคิวงานที่ต้องให้บริการตามลำดับการมาถึง

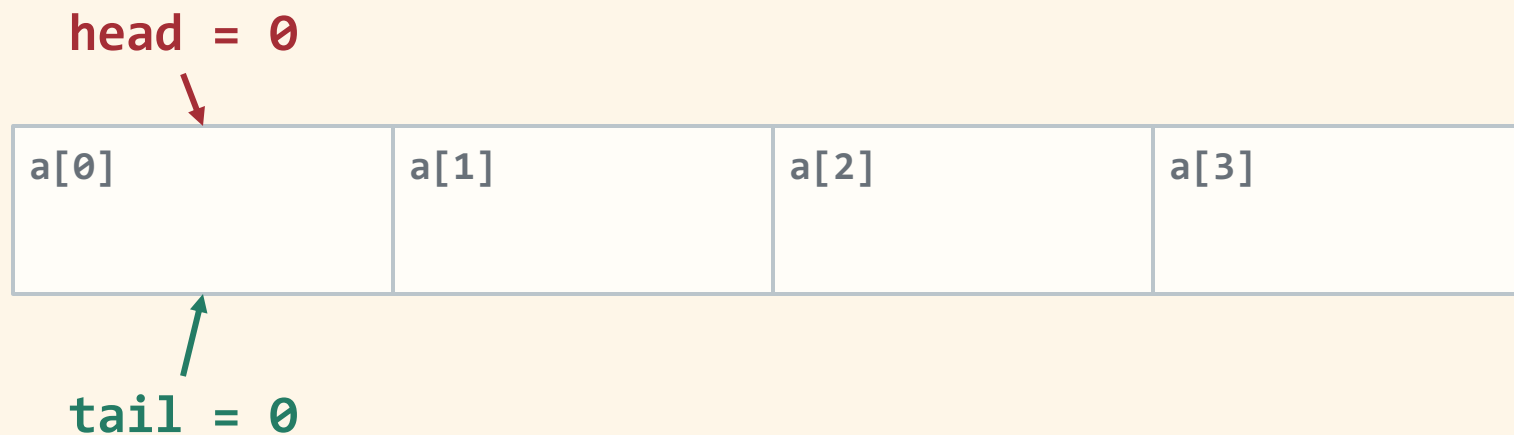




หัวคิวและตำแหน่งเพิ่มข้อมูลครั้งถัดไป

ในตัวอย่างอาร์เรย์นี้ head เก็บ index ของสมาชิกหัวคิว ส่วน tail เก็บ index ของช่องที่จะใช้เขียนข้อมูลตัวถัดไป เราเริ่มคิวว่างด้วย $\text{head} = \text{tail} = 0$

เราจะใช้ข้อตกลงนี้ตลอดภาพชุดต่อไป จึงต้องระวังว่า tail ไม่ได้เก็บ index ของสมาชิกตัวท้ายที่มีข้อมูลอยู่

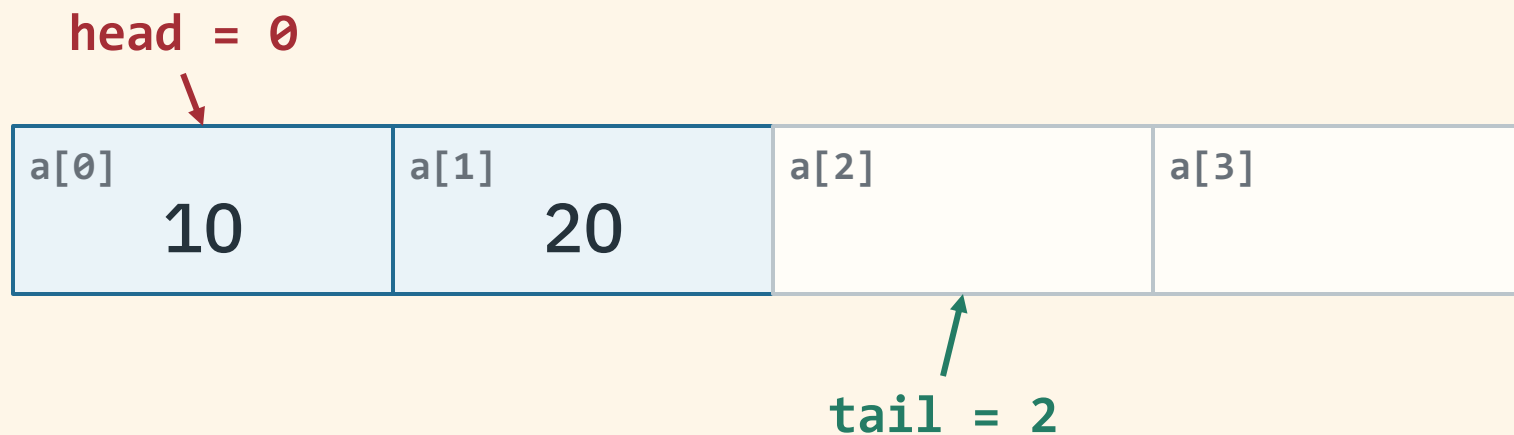


head และ tail เท่ากัน จึงยังไม่มีสมาชิกในคิว



เมื่อเพิ่มข้อมูล ท้ายคิวจะขยับไปข้างหน้า

หลังเพิ่ม 10 และ 20 ข้อมูลอยู่ที่ $a[0]$ และ $a[1]$ ส่วน $tail$ เลื่อนไปเป็น 2 เพื่อรอการเพิ่มครั้งถัดไป
สมาชิกหัวคิวยังคงเป็น 10 เพราะ $head$ ยังเป็น 0



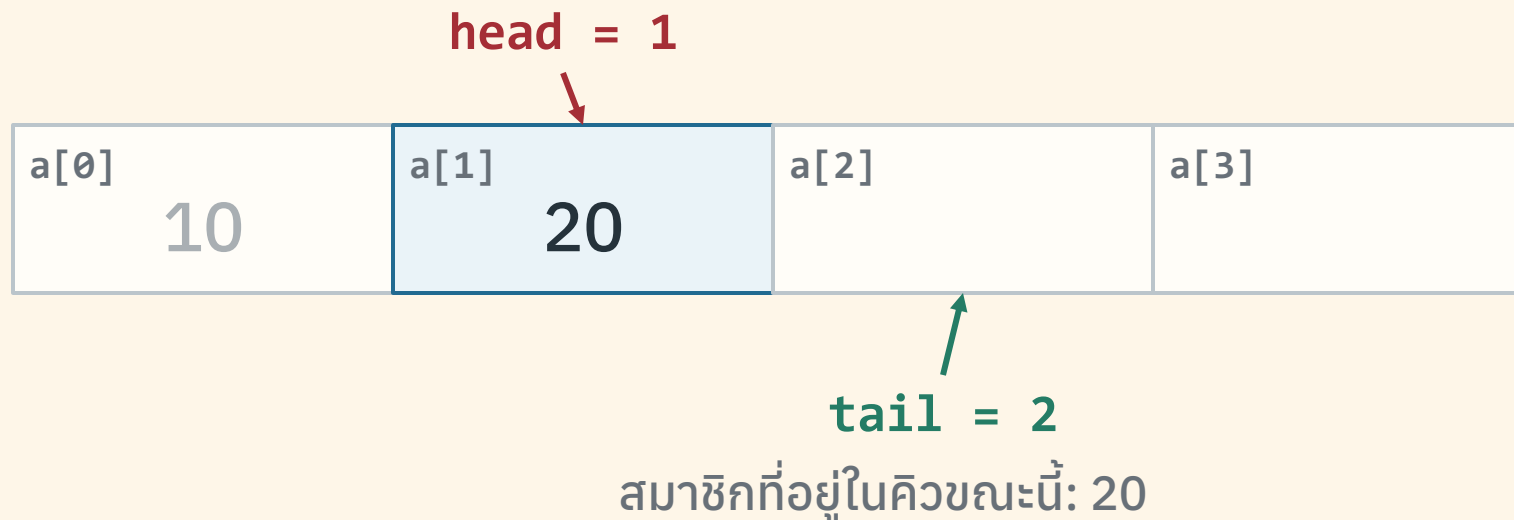
สมาชิกที่อยู่ในคิวขณะนี้: $10 \rightarrow 20$



เมื่อนำหัวคิวออก เราเลื่อน Head

เมื่อนำค่า 10 ออกจากคิว head จะเลื่อนไปเป็น 1 สมาชิกหัวคิวตัวใหม่จึงเป็น 20

เราไม่ต้องเลื่อนข้อมูลทุกตัวในอาร์เรย์ เพียงเปลี่ยนว่าช่วงข้อมูลที่ยังเป็นสมาชิกเริ่มต้นตรงไหน

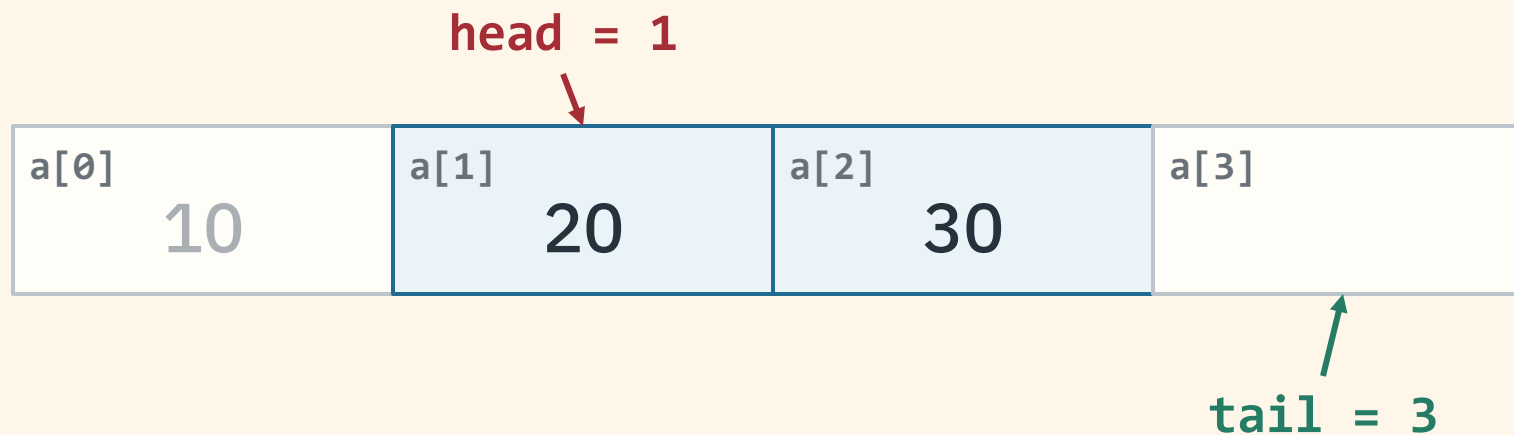




ข้อมูลใหม่ยังเข้าที่ท้ายคิว

เมื่อเพิ่ม 30 หลังจากนั้น เราเก็บข้อมูลที่ $a[2]$ แล้วเลื่อน $tail$ เป็น 3 ลำดับในคิวจึงเป็น 20 แล้วตามด้วย 30

ในตัวอย่างนี้ ช่วงสมาชิกที่ใช้งานคือ index ตั้งแต่ $head$ จนถึงก่อน $tail$ และคิวจะว่างเมื่อทั้งสองค่าเท่ากัน



สมาชิกที่อยู่ในคิวขณะนี้: $20 \rightarrow 30$



อาร์เรย์แบบง่ายยังไม่วนกลับมาใช้ช่องเดิม

การใช้ head และ tail แบบนี้ทำให้เพิ่มท้ายและนำหัวออกได้ง่าย แต่ช่องที่อยู่ก่อน head ยังไม่ถูกนำกลับมาใช้

ถ้าจำนวนครั้งที่เพิ่มข้อมูลมีขอบเขตแน่นอน เราเตรียมอาร์เรย์ให้พอได้ หรือแก้ปัญหาด้วยการใช้แบบ circular queue

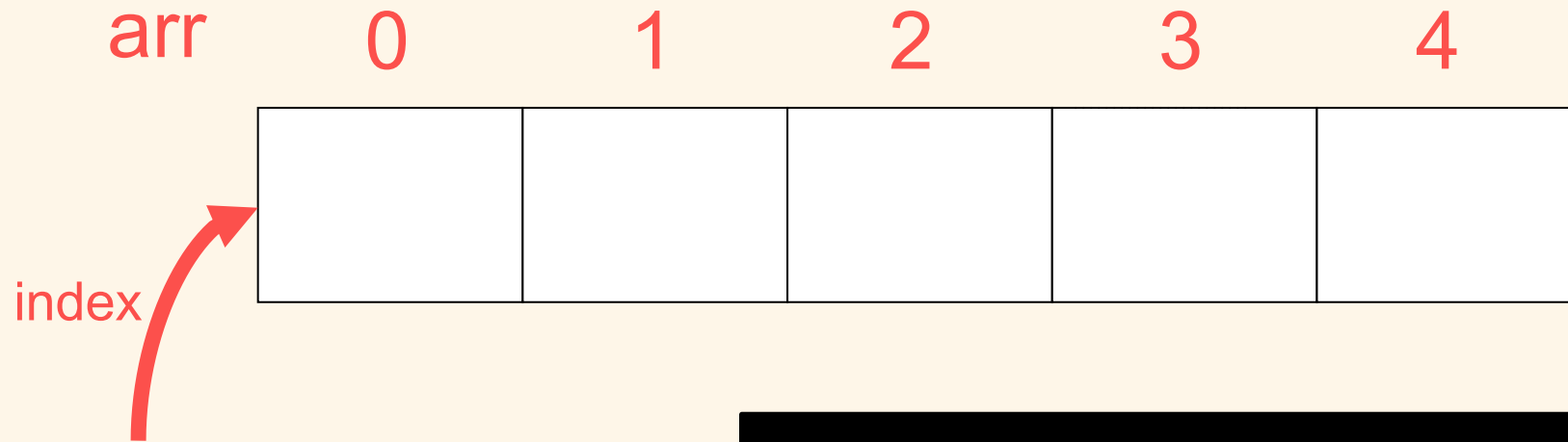
วันนี้เราจะใช้ `std::queue` ในโจทย์

```
// After checking capacity  
a[tail++] = value;
```

```
// When head < tail  
int first = a[head++];
```



Queue



front = -1

rear = -1

```
arr[front]  
arr[rear]
```



Queue

arr

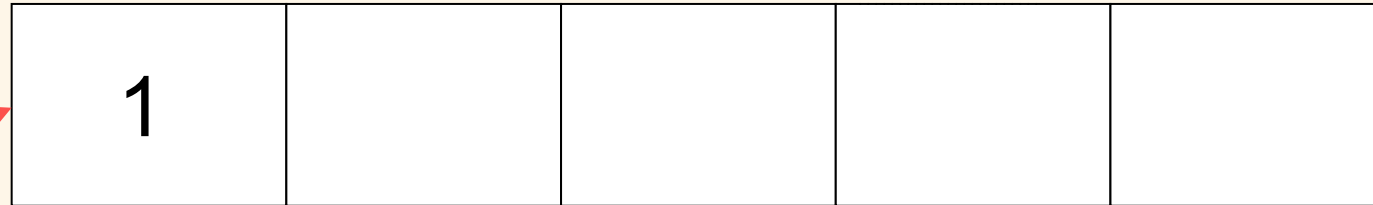
0

1

2

3

4



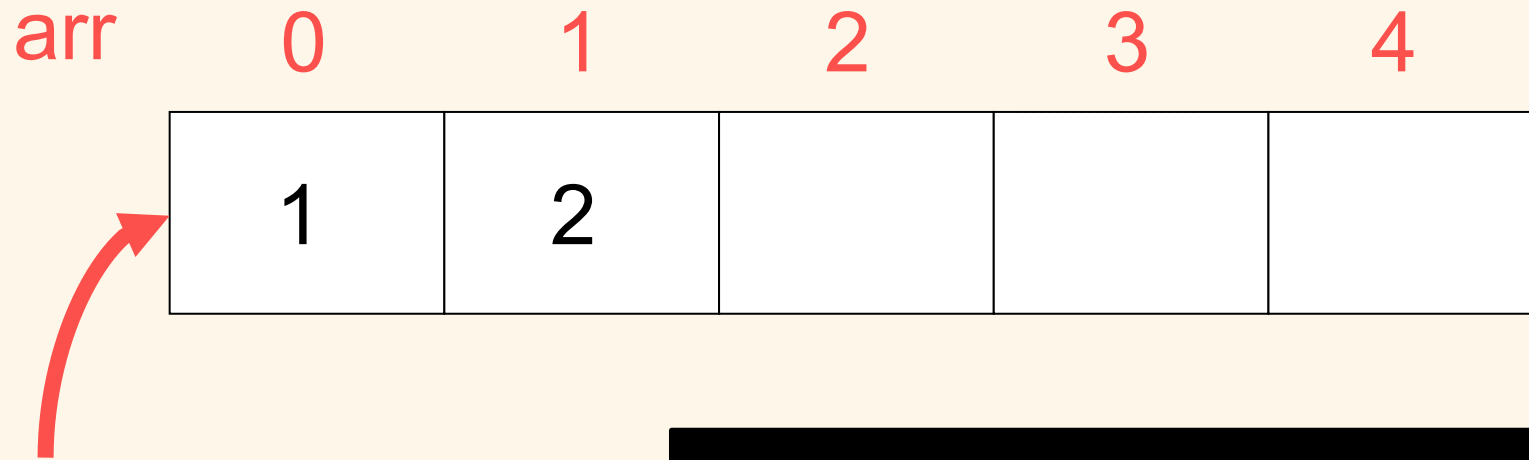
front = 0

rear = 0

```
arr[front]  
arr[rear]
```



Queue



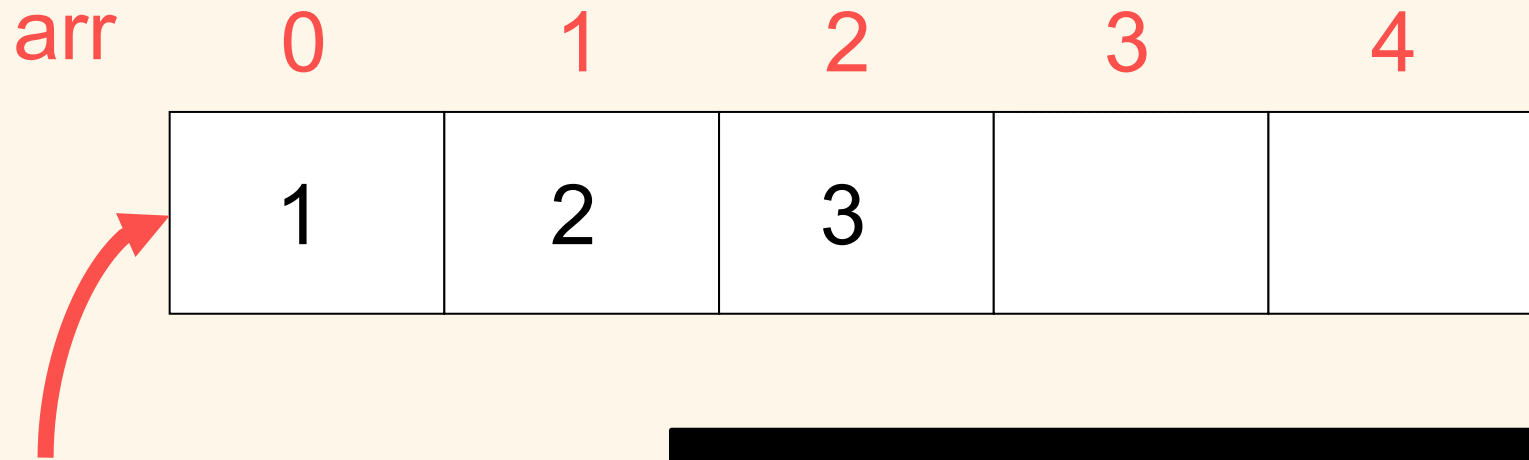
front = 0

rear = ~~0~~ 1

```
arr[front]
arr[rear]
```



Queue



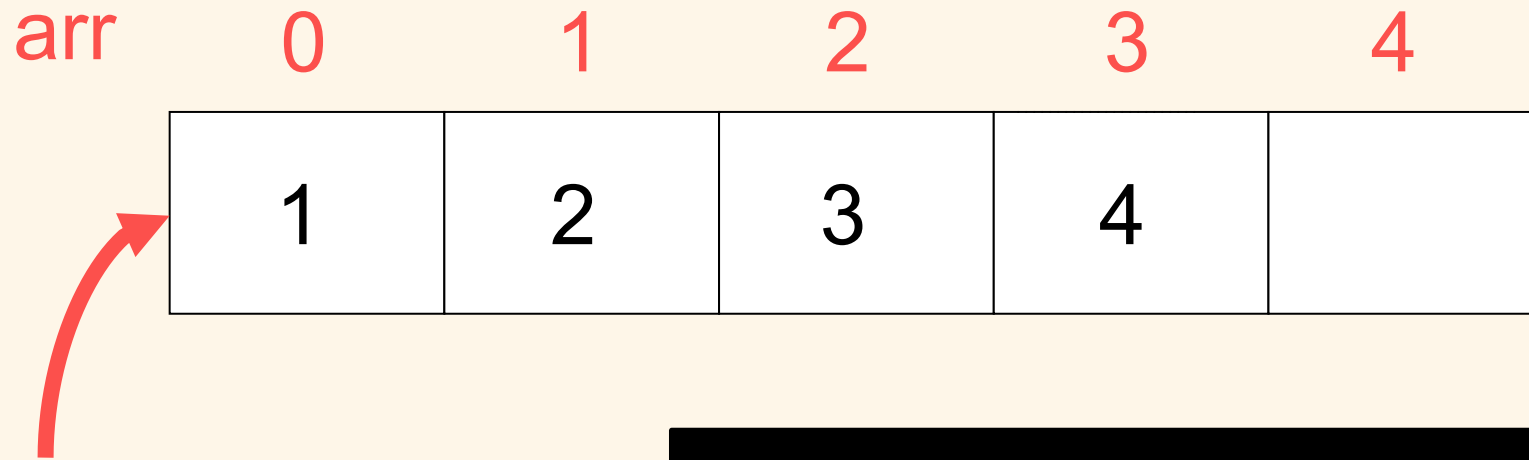
front = 0

rear = ~~0~~ * 2

```
arr[front]
arr[rear]
```



Queue



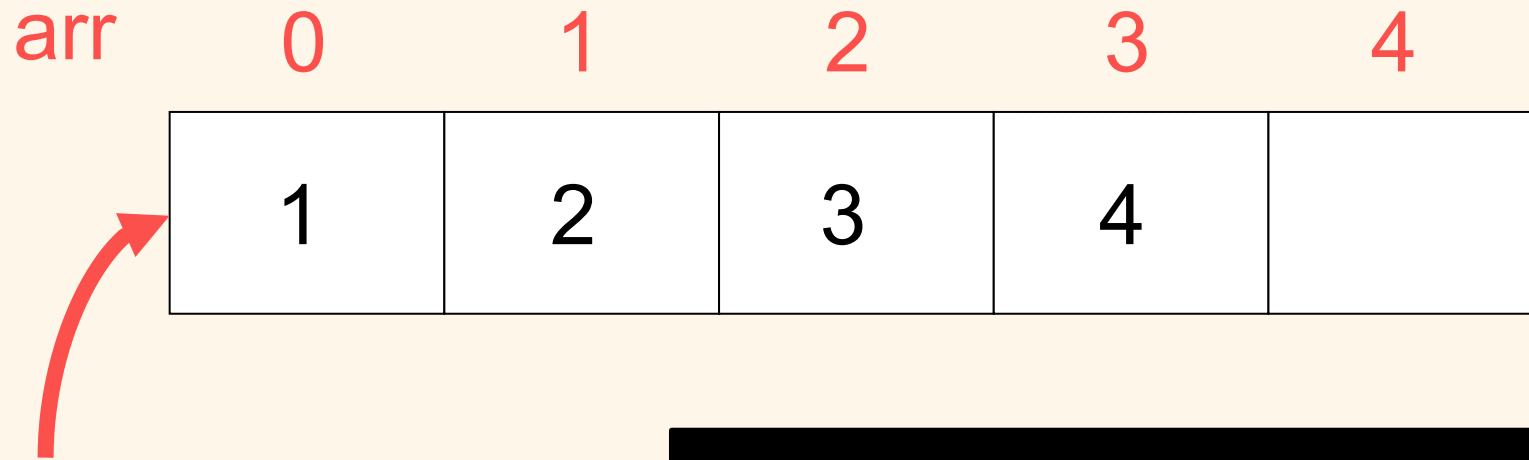
front = 0

rear = ~~0~~ ~~1~~ ~~2~~ 3

```
arr[front]
arr[rear]
```



Queue



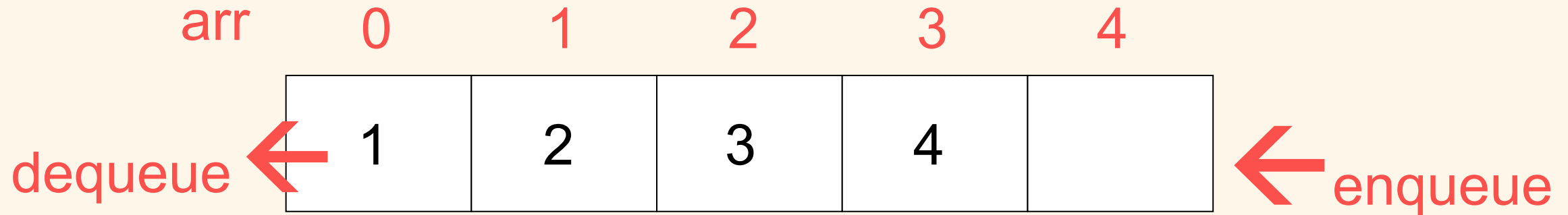
front = 0

rear = ~~0~~ ~~1~~ ~~2~~ 3

arr[front] → front
arr[rear] → rear, last



Queue



front = 0

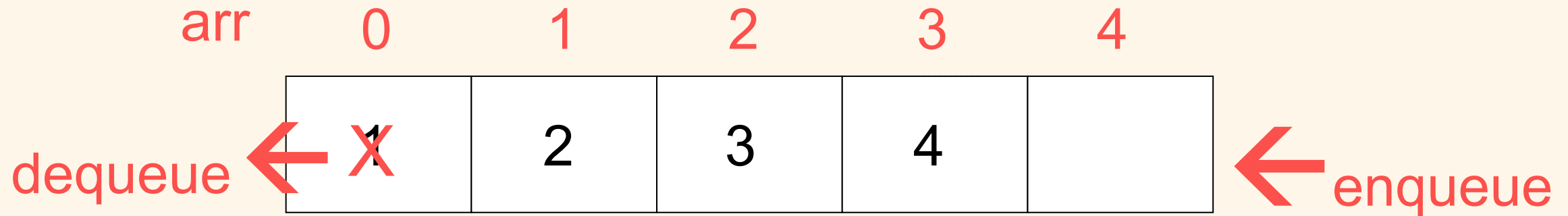
rear = ~~0~~ ~~1~~ ~~2~~ 3

arr[front] → front

arr[rear] → rear, last



Queue



front = ~~0~~ 1

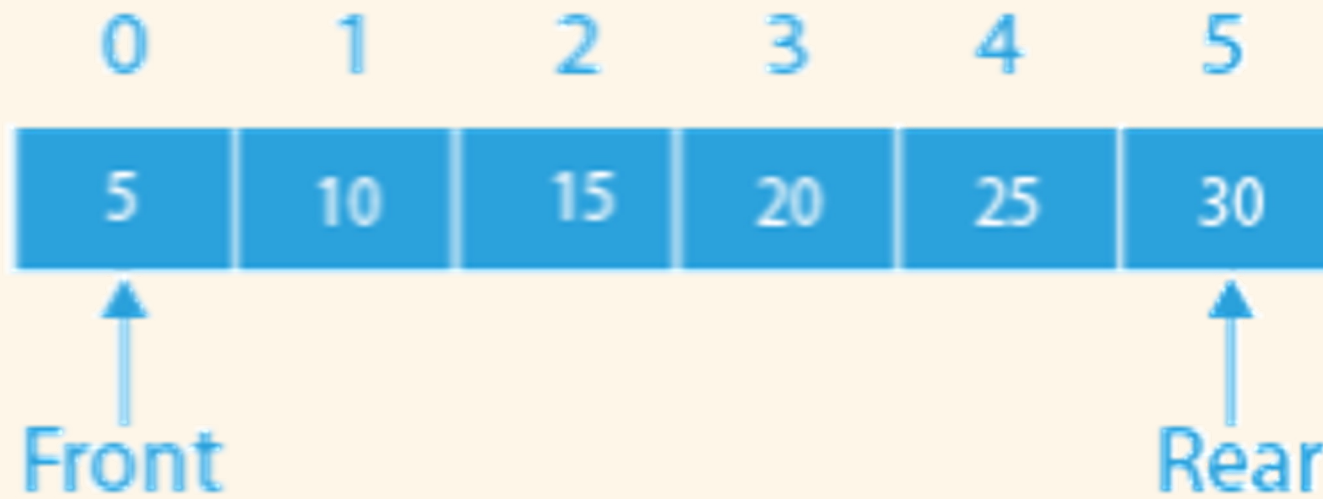
rear = ~~0~~ ~~1~~ ~~2~~ 3

arr[front] → front

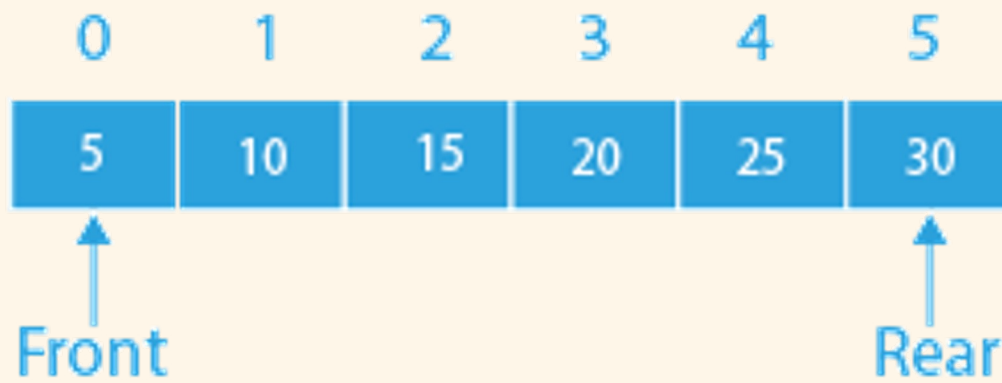
arr[rear] → rear, last



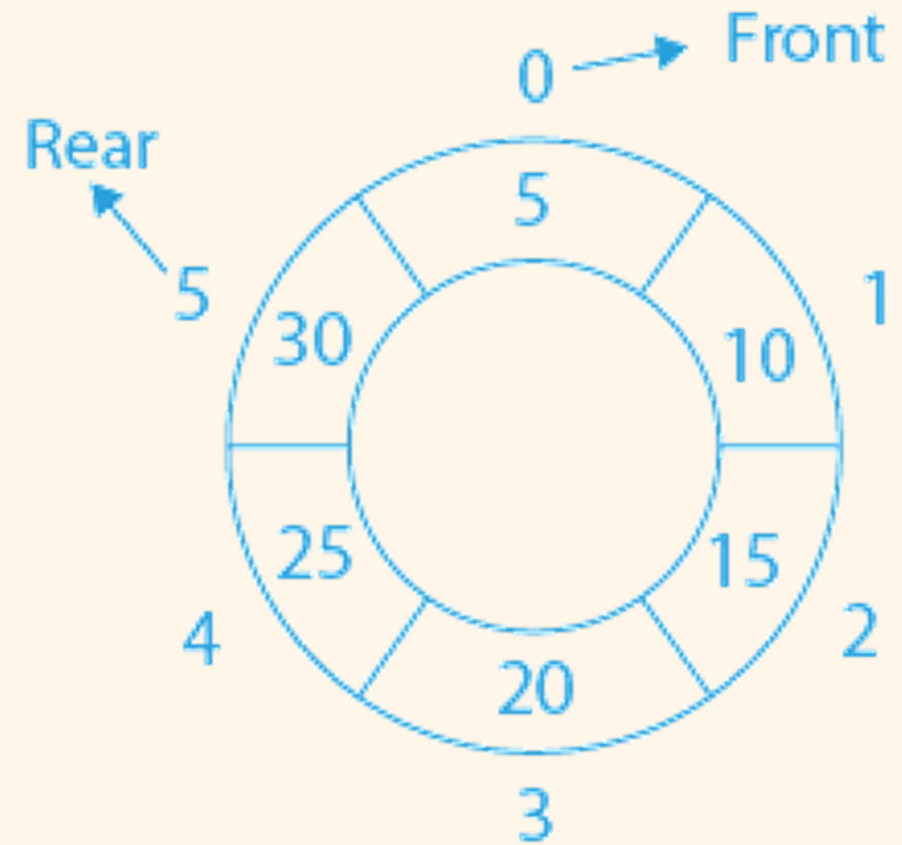
1. **Linear Queue** เป็น Queue ที่ข้อมูลจะถูกเพิ่ม (enqueue) เข้าไปทางด้านหลัง และลบ (dequeue) ออกจากด้านหน้า



Linear Queue



Linear Queue

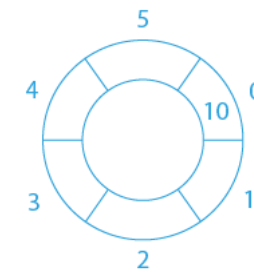


Circular Queue

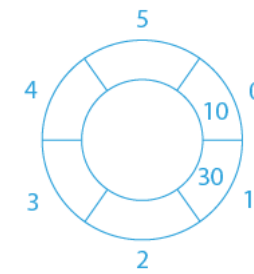


2. Circular Queue Queue ที่เพิ่มประสิทธิภาพ โดยการเชื่อมตำแหน่งท้ายสุดเข้ากับตำแหน่งแรก

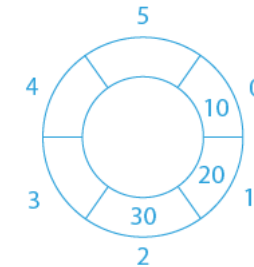
เป็นการแก้ปัญหาพื้นที่เหลือทิ้งโดยการนำพื้นที่ที่
เกิดจากการลบข้อมูลกลับมาใช้ใหม่ได้



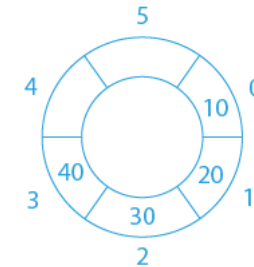
Enqueue (10)



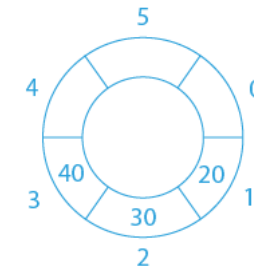
Enqueue (20)



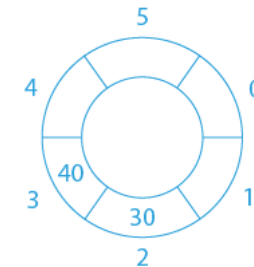
Enqueue (30)



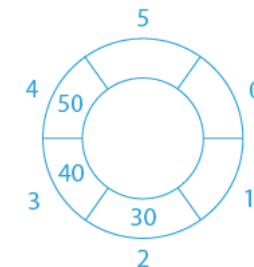
Enqueue (40)



Enqueue ()



Enqueue ()



Enqueue (50)

Queue

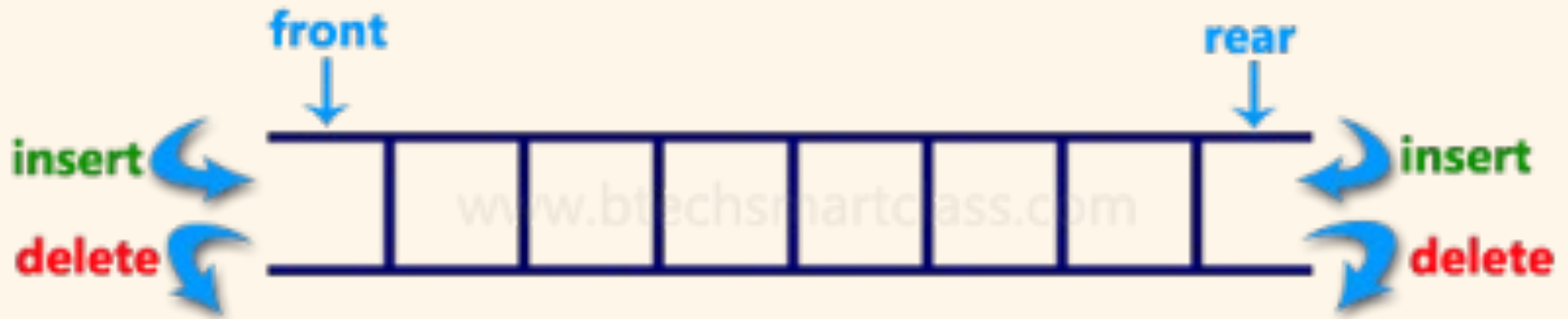


3. **Priority Queue** abstract data type ที่แต่ละข้อมูลจะมีค่าความสำคัญ (priority) กำกับไว้ การลบข้อมูล (dequeue) จะลบข้อมูลที่มีค่าความสำคัญสูงสุดออกไปก่อน

	Rear ↓							
Priority	3	2	3	2	1			
Element	10	20	30	40	50			
Index	0	1	2	3	4	5	6	7

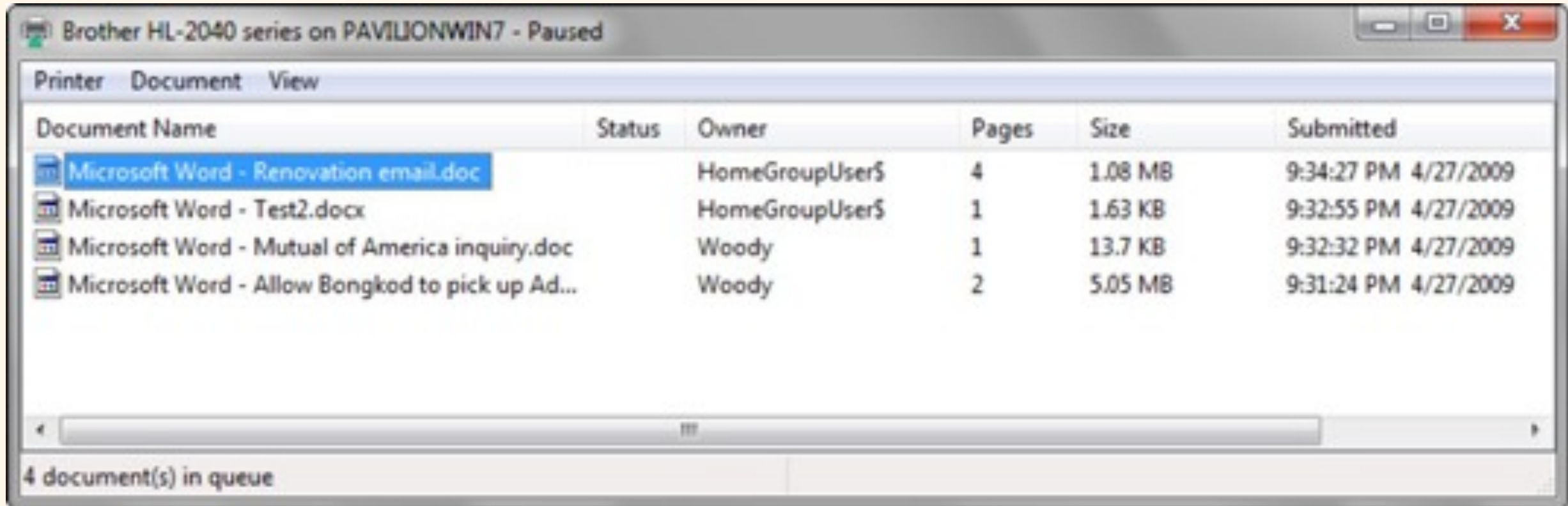


4. Double-Ended Queue (Deque) Queue ที่มีความยืดหยุ่น ในการเพิ่ม (insert) หรือลบ (delete) ข้อมูลได้ทั้งจากด้านหน้าและด้านหลัง





สามารถเจอ Queue ได้จากไหน?





std::queue ใช้กติกาเดียวกับภาพที่เราเรียน

std::queue เตรียมคำสั่งเพิ่มท้ายและนำหัวออกไว้ให้แล้ว เราจึงมุ่งที่กติกาของโจทย์ได้โดยไม่ต้องจัดการตำแหน่งในอาร์เรย์เอง

front() และ back() ใช้เข้าถึงสมาชิก ส่วน pop() นำหัวคิวออกโดยไม่คืนข้อมูล เราต้องตรวจสอบว่าคิวไม่ว่างก่อนเข้าถึงสมาชิก

คำสั่ง	ความหมาย
push(x)	เพิ่ม x ที่ท้ายคิว
front() และ back()	เข้าถึงสมาชิกหัวคิวและท้ายคิว
pop()	นำสมาชิกหัวคิวออก โดยไม่คืนค่า
empty() และ size()	ตรวจสอบว่างและดูจำนวนสมาชิก



Queue เก็บข้อมูลเป็น Struct ได้

งานหนึ่งชิ้นอาจมีทั้งหมายเลขและปริมาณงาน
ที่เหลือ เราจึงใช้ struct Job รวมข้อมูลสอง
ส่วนนี้ แล้วเก็บ Job ลงใน queue

เมื่อถึงคิว เรารับสำเนาจาก front() มาก่อน
แล้วจึง pop() งานนั้นออกเพื่อประมวลผลต่อ

```
#include <queue>

struct Job {
    int id;
    int remaining;
};

std::queue<Job> jobs;
jobs.push({1, 5});

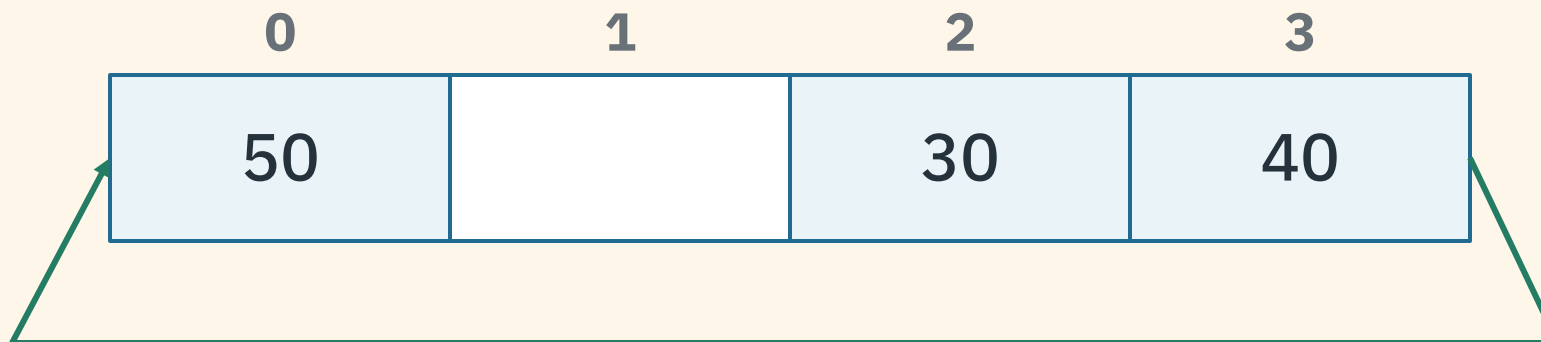
Job job = jobs.front();
jobs.pop();
```



Circular queue นำช่องที่ว่างกลับมาใช้

อาร์เรย์ของ circular queue สามารถเลื่อนตำแหน่งกลับมาที่ต้นอาร์เรย์เมื่อถึงท้าย ทำให้ช่องที่ว่างจากการนำข้อมูลออกกลับมาใช้งานได้

สิ่งที่ต้องกำหนดให้ชัดคือวิธีแยกสถานะว่างออกจากสถานะเต็ม เช่น เก็บจำนวนสมาชิกเพิ่ม หรือเว้นหนึ่งช่องไว้ตามข้อตกลง



ตัวอย่างลำดับในคิว: 30 → 40 → 50



Challenge 4: หุ่นยนต์ผลัดกันทำงาน

หุ่นยนต์หมายเลข 1 ถึง n เข้าแถวตามลำดับ แต่ละตัวมีงานที่ต้องทำ เมื่อถึงหัวคิว หุ่นยนต์จะทำงาน k หน่วย หรือทำงานที่เหลือทั้งหมดหากเหลือน้อยกว่า k

ถ้ายังทำไม่เสร็จ หุ่นยนต์ต้องกลับไปต่อท้ายคิว แต่ถ้าเสร็จแล้วให้ออกจากระบบและรายงานหมายเลขงานลำดับที่หุ่นยนต์ทำงานเสร็จ

1: เหลือ 5

หุ่นยนต์ 1

2: เหลือ 2

หุ่นยนต์ 2

3: เหลือ 4

หุ่นยนต์ 3

เมื่อ $k = 2$ ลำดับที่ทำงานเสร็จคือ $2 \rightarrow 3 \rightarrow 1$



ลองจำลองคิวก่อนเขียน Challenge 4

รับ n และ k ตามด้วยปริมาณงานของหุ่นยนต์ n ตัว แล้วแสดงหมายเลขตามลำดับที่ทำเสร็จ โดยคั่นด้วยช่องว่าง

ตัวอย่าง: หุ่นยนต์ 1, 2 และ 3 มีงาน 5, 2 และ 4 หน่วย เมื่อ $k = 2$ หุ่นยนต์ 1 จะเหลืองาน 3 หน่วยและกลับท้ายคิว ลองเดินต่อจนได้คำตอบ 2 3 1

สิ่งที่กำหนด	ขอบเขตหรือความหมาย
n และงานเริ่มต้น	อยู่ในช่วง 1–1000
k	อยู่ในช่วง 1–1000
งานเหลือไม่เกิน k	ทำเสร็จในรอบนี้และไม่กลับเข้าคิว

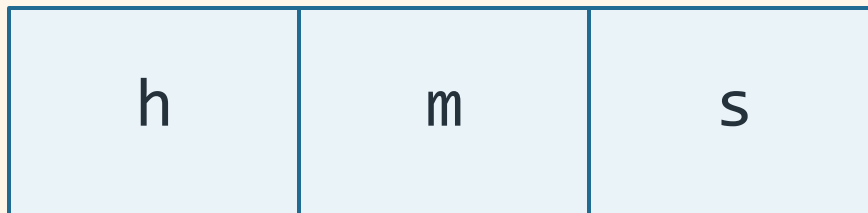


Struct และ Union เก็บข้อมูลต่างกันอย่างไร

Struct เหมาะเมื่อเราต้องเก็บสมาชิกหลายส่วนไว้พร้อมกัน เช่น ชั่วโมง นาที และวินาทีของเวลาเดียวกัน

Union ให้สมาชิกใช้พื้นที่ร่วมกัน โดยในตัวอย่างนี้เราจะใช้งานสมาชิกเพียงหนึ่งตัวในขณะหนึ่ง ผู้เขียนจึงต้องรู้ว่าตอนนี้ควรอ่านสมาชิกใด

Struct: เก็บพร้อมกัน



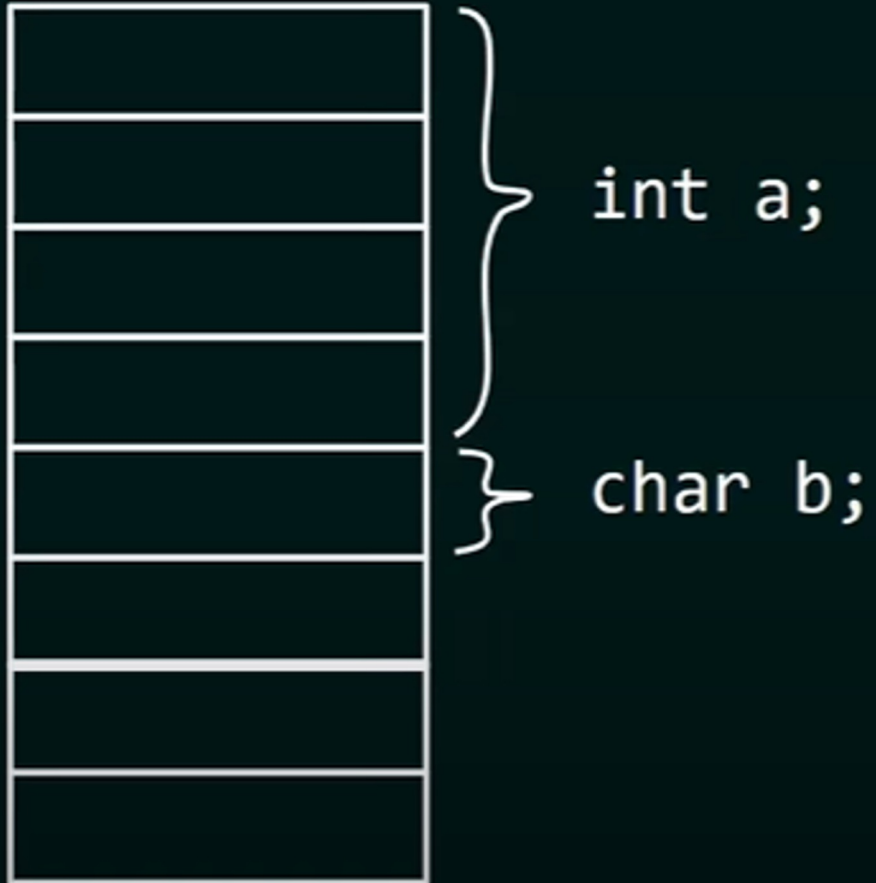
Union: ใช้พื้นที่ร่วมกัน



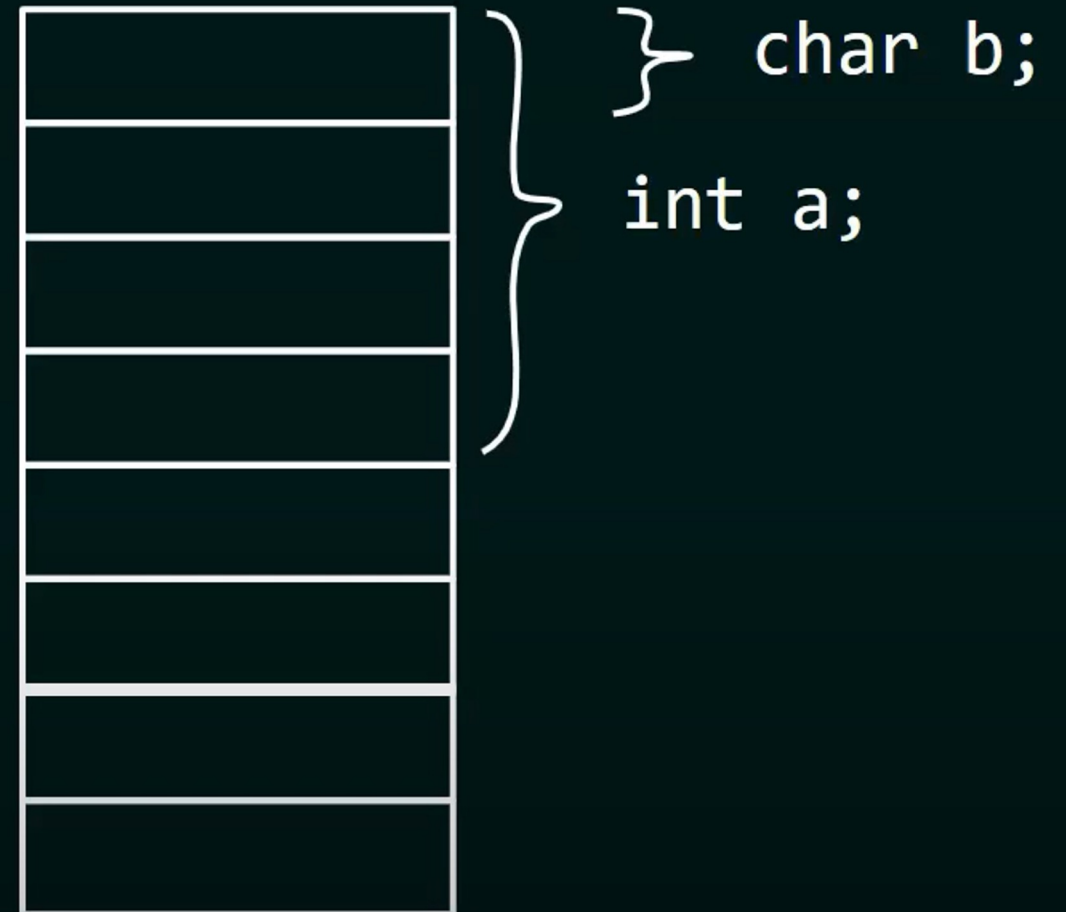
ภาพนี้แสดงแนวคิด ไม่ได้กำหนดจำนวนไบต์จริง



struct abc



union abc





```
#include <iostream>
using namespace std;
```

```
// การกำหนด Union
```

```
union Data {
    int intValue;
    char charValue;
    float floatValue;
};
```

```
// ฟังก์ชันหลัก
```

```
int main()
```

```
{
```

```
    // การกำหนดค่าให้กับ Union
```

```
    union Data data;
```

```
    data.intValue = 34;
```

```
    // แสดงผลค่าที่ถูกจัดเก็บในหน่วยความจำ
```

```
    cout << "The first value at the allocated memory: " << data.intValue << endl;
```

```
    data.charValue = 'A';
```

```
    cout << "The next value stored after removing the previous value: " << data.charValue << endl;
```

```
    data.floatValue = 34.34;
```

```
    cout << "The Final value value at the same allocated memory space: " << data.floatValue << endl;
```

```
    return 0;
```

```
}
```



```
#include <iostream>
using namespace std;
```

```
// การกำหนด Union
```

```
union Data {
    int intValue;
    char charValue;
    float floatValue;
```

```
// ฟังก์ชันหลัก
```

```
int main()
```

```
{
```

```
    // การกำหนดค่าให้กับ Union
```

```
    union Data data;
```

```
    data.intValue = 34;
```

```
    // แสดงผลค่าที่ถูกจัดเก็บในหน่วยความจำ
```

```
    cout << "The first value at the allocated memory: " << data.intValue << endl;
```

เมื่อเรากำหนดค่า 'A' ให้กับตัวแปร **charValue** ซึ่งเป็นประเภท **char** ค่าเก่าที่ถูกเก็บไว้ใน **intValue** (ค่า 34) จะถูกลบออกและแทนที่ด้วยค่าใหม่ที่เป็นตัวอักษร 'A' ซึ่งถูกเก็บในพื้นที่หน่วยความจำเดียวกันกับ **intValue** และสามารถแสดงผลอย่างถูกต้องใน **Output**

The first value at the allocated memory : 34

The next value stored after removing the previous value : A

The Final value value at the same allocated memory space : 34.34



File In C++

เอกสารประกอบการอบรม

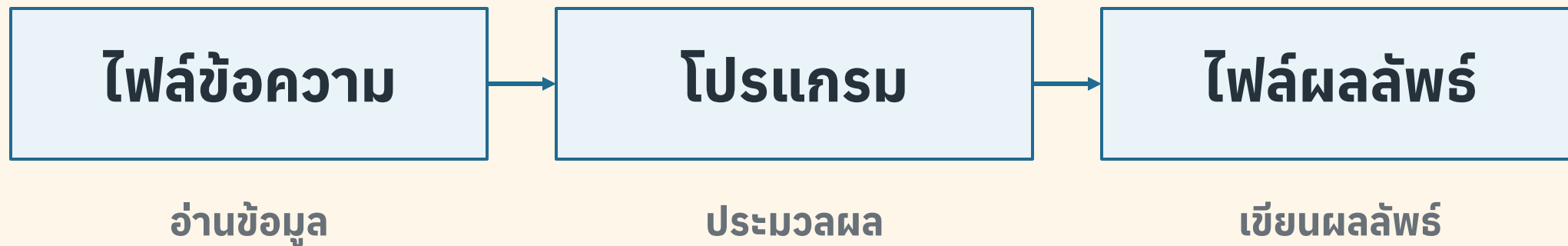
สอวน. สาขาคอมพิวเตอร์ ศูนย์โรงเรียนสวนกุหลาบวิทยาลัย นนทบุรี



ไฟล์ทำให้ข้อมูลยังอยู่หลังโปรแกรมจบ

ที่ผ่านมาเราอ่านข้อมูลจากแป้นพิมพ์และแสดงผลบนหน้าจอ ไฟล์ช่วยให้เราเก็บข้อมูลไว้เพื่อนำกลับมาอ่านในครั้งต่อไปได้

ในบทนี้เราจะใช้ไฟล์ข้อความที่มีรูปแบบแน่นอน โดยอ่านข้อมูลที่ละเอียดเบี่ยน ประมวลผล แล้วเขียนผลลงไฟล์อีกไฟล์หนึ่ง





รูปแบบไฟล์ต้องชัดเจนก่อนเริ่มอ่าน

ไฟล์ scores.txt ของเราจะเก็บชื่อหนึ่งคำกับคะแนนจำนวนเต็มในแต่ละระเบียน ดังนั้นโปรแกรมต้องอ่านข้อมูลทั้งสองส่วนให้ครบก่อนนำไปใช้

หากจะเก็บชื่อที่มีช่องว่าง เราต้องกำหนดรูปแบบและวิธีอ่านใหม่ให้เหมาะสม วันนี้จึงใช้ชื่อคำเดียวก่อน

ตัวอย่างข้อมูลใน scores.txt

```
Mali 80  
Nok 70  
Beam 90
```



เปิดไฟล์แล้วตรวจว่าเปิดสำเร็จหรือไม่

เราใช้ ifstream สำหรับอ่านไฟล์ และตรวจสถานะก่อนเริ่มอ่าน หากเปิดไม่สำเร็จ ควรแจ้งข้อผิดพลาดแล้วหยุดการทำงานส่วนนั้น

ชื่อไฟล์แบบ scores.txt อ้างอิงจาก working directory ของโปรแกรม ซึ่งอาจไม่ใช่โฟลเดอร์เดียวกับไฟล์ซอร์สโค้ด

```
#include <iostream>
#include <fstream>

std::ifstream in("scores.txt");
if (!in) {
    std::cerr << "Cannot open file";
    return 1;
}
```



อ่านสำเร็จครบหนึ่งระเบียนแล้วจึงประมวลผล

เงื่อนไขของ while จะเป็นจริงเมื่ออ่านชื่อและคะแนนได้ครบ เราจึงใช้ข้อมูลในรอบนั้นได้อย่างมั่นใจ

เมื่ออ่านต่อไม่ได้ loop จะหยุดเอง เราไม่ใช้ while (!in.eof()) เพื่อเดาว่าการอ่านครั้งถัดไปจะสำเร็จหรือไม่

```
std::string name;  
int score;  
  
while (in >> name >> score) {  
    std::cout << name << " "  
                << score << "\n";  
}
```



การเขียนใหม่และการต่อท้ายไฟล์

เมื่อเปิด ofstream แบบปกติสำเร็จ เนื้อหาเดิมของไฟล์จะถูกลบเพื่อเริ่มเขียนใหม่ แม้เรายังไม่ได้เขียนข้อมูลลงไป ส่วน std::ios::app ใช้เพิ่มข้อมูลต่อท้าย

ก่อนเขียน เราควรตรวจสอบว่าเปิดไฟล์ได้ และเมื่อเขียนเสร็จต้องปิดไฟล์หรือปล่อยให้ stream ปิดเมื่อออกจากขอบเขตการใช้งาน

```
std::ofstream out("summary.txt");  
  
// Append to an existing log  
std::ofstream log(  
    "history.txt", std::ios::app  
);
```



เราต้องตรวจสอบผลของการเขียนด้วย

เมื่อเปิดไฟล์ได้แล้ว เราเขียนข้อมูลด้วย <<
คล้ายกับ cout จากนั้นปิดไฟล์เพื่อให้ข้อมูลที่
ค้างอยู่ถูกส่งออกไป

การเปิดสำเร็จไม่ได้แปลว่าการเขียนทุกครั้งจะ
สำเร็จเสมอ ดังนั้นถ้าต้องรายงานว่างานเสร็จ
แล้ว เราควรตรวจสอบสถานะหลังเขียนและปิดด้วย

```
if (!out) {  
    return 1;  
}  
out << count << " " << total  
    << "\n";  
out.close();  
if (!out) {  
    return 1;  
}
```



เมื่ออ่านไฟล์ไม่ได้ ควรตรวจสอบไหนก่อน

เริ่มจากตรวจสอบชื่อไฟล์และ working directory แล้วเปิดดูว่าข้อมูลจริงตรงกับรูปแบบที่โปรแกรมคาดไว้หรือไม่

อีกประเด็นหนึ่งคือไฟล์ที่ใช้เขียนสาริตต้องตรงกับไฟล์ที่ใช้สาริตการอ่าน หากเขียนประโยคทั่วไปไว้ โปรแกรมที่คาดหวังชื่อกับคะแนนก็อ่านไม่ครบ

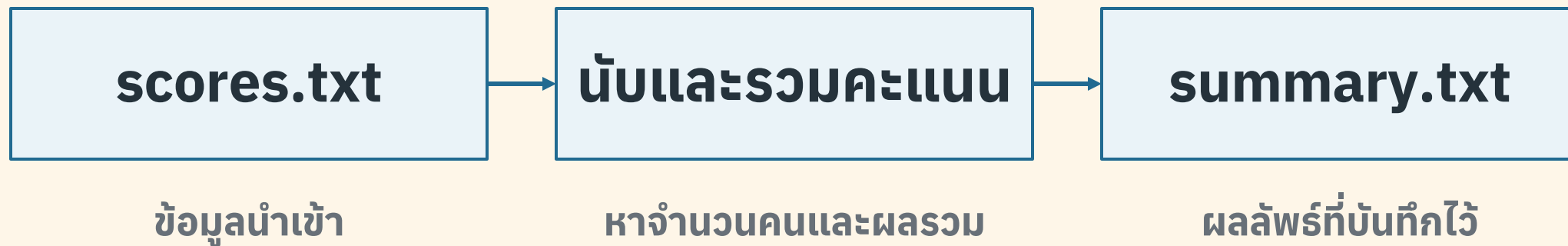
อาการ	สิ่งที่ควรตรวจสอบ
เปิดไฟล์ไม่สำเร็จ	ชื่อไฟล์ ตำแหน่งไฟล์ และสิทธิ์เข้าถึง
เปิดได้แต่อ่านข้อมูลไม่ครบ	รูปแบบชื่อและชนิดของคะแนน
รันอีกครั้งแล้วข้อมูลเดิมหาย	โหมดเขียนใหม่หรือโหมดต่อท้าย



Challenge 5: สรุปคะแนนจากไฟล์

ให้อ่าน scores.txt ซึ่งแต่ละระเบียนมีชื่อหนึ่งคำและคะแนน แล้วหาจำนวนคนกับผลรวมคะแนนทั้งหมด จากนั้นเขียนผลลง summary.txt

กำหนดว่าคะแนนอยู่ในช่วง 0–100 และข้อมูลถูกต้องตามรูปแบบ หากไฟล์ว่างให้เขียน 0 0 แต่หากเปิดไฟล์ไม่ได้ให้แจ้งข้อผิดพลาด





ผลลัพธ์ของ Challenge 5 ต้องอยู่ในไฟล์

ไฟล์มีได้ไม่เกิน 100000 ระเบียบ ให้ใช้ long long เก็บผลรวม และเขียนจำนวนคนกับผลรวมคั่นด้วยช่องว่างหนึ่งบรรทัด ดังตัวอย่างทางขวา

เริ่มจากเปิดอ่านและแสดงข้อมูลให้ถูกต้องก่อน แล้วจึงเพิ่มการนับและหาผลรวม สุดท้ายให้เปิดไฟล์ผลลัพธ์ตรวจด้วยตนเอง

scores.txt

Mali 80

Nok 70

Beam 90

summary.txt

3 240



```
#include <iostream>
#include <fstream>

using namespace std;

int main()
{
    ofstream myFile("score.txt", ios::out);

    if (!myFile) { // ตรวจสอบว่าเปิดไฟล์สำเร็จหรือไม่
        cout << "Error opening file!" << endl;
        return 1;
    }

    // เขียนข้อมูลไปที่ไฟล์
    myFile << "This is a test score." << endl;

    myFile.close(); // ปิดไฟล์

    return 0;
}
```



รับข้อมูลจากผู้ใช้ และเขียนไฟล์

```
#include <iostream>
#include <fstream>
#include <cstring>

using namespace std;

int main()
{
    ofstream myFile("score.txt", ios::out);
    char name[30];
    int score; // เพิ่มเครื่องหมาย ;

    cout << "Enter name and score (type 'exit' as name to stop): " << endl;
    while (true)
    {
        cin >> name;
        if (strcmp(name, "exit") == 0) // ถ้าพิมพ์ 'exit' จะหยุดการป้อนข้อมูล
            break;

        cin >> score; // อ่านคะแนน
        myFile << name << '\t' << score << '\n'; // เขียนชื่อและคะแนนไปที่ไฟล์
    }

    myFile.close(); // ปิดไฟล์

    cout << "Data saved successfully." << endl;

    return 0;
}
```



```
#include <iostream>
#include <fstream>

using namespace std;

int main()
{
    ifstream readFile("score.txt", ios::in);
    char name[30];
    int score;

    // ตรวจสอบว่าเปิดไฟล์สำเร็จหรือไม่
    if (!readFile) {
        cout << "Error opening file!" << endl;
        return 1; // ออกจากโปรแกรมหากไม่สามารถเปิดไฟล์
    }

    // อ่านข้อมูลจากไฟล์
    while (readFile >> name >> score) {
        cout << name << ' ' << score << endl; // แสดงชื่อและคะแนน
    }

    readFile.close(); // ปิดไฟล์
    return 0;
}
```



เลือกโครงสร้างจากกฏิติกาของงาน

ก่อนเขียนโปรแกรม ลองถามว่างานต้องการเข้าถึงหรือเปลี่ยนข้อมูลอย่างไร ถ้าต้องย้อนสิ่งล่าสุดให้คิดถึง stack ถ้าต้องรักษาลำดับการรอให้คิดถึง queue

Linked list มีประโยชน์เมื่อเราต้องจัดการการเชื่อมโยง แต่การหาตำแหน่งยังอาจต้องเดินตามโหนด ไม่มีโครงสร้างใดเหมาะที่สุดกับทุกงาน

ลักษณะงาน	แนวคิดที่เกี่ยวข้อง
รวมข้อมูลของสิ่งเดียวกัน	Struct
ย้อนสิ่งที่เพิ่งเกิดล่าสุด	Stack
ให้บริการตามลำดับที่เข้ามา	Queue
เดินหรือเปลี่ยนลำดับตามลิงก์	Linked list



Challenge 6: วงเล็บถูกคู่

รับข้อความหนึ่งบรรทัดที่มีเฉพาะวงเล็บ () [] { } จงตรวจสอบว่าวงเล็บเปิดและปิดจับคู่กันถูกต้องและซ้อนกันถูกต้องหรือไม่

ให้แสดง YES เมื่อถูกต้อง และ NO เมื่อไม่ถูกต้อง ความยาวข้อความอยู่ในช่วง 1–100000 ตัวอักษรเริ่มจากวงเล็บ () ชนิดเดียวก่อนแล้วจึงขยายเป็นสามชนิด

([])

ตอบ YES

([)]

ตอบ NO



จำนวนวงเล็บเท่ากัน อาจยังจับคู่ไม่ถูก

ข้อความ ([)] มีจำนวนวงเล็บเปิดและปิดเท่ากัน แต่ลำดับการซ้อนไม่ถูกต้อง ส่วน ([]) จับคู่ได้ถูกต้อง
ก่อนเริ่มเขียน ลองอธิบายว่าวงเล็บปิดหนึ่งตัวควรจับคู่กับวงเล็บเปิดตัวใด และเมื่ออ่านข้อความจบแล้วควร
เหลืออะไรในโครงสร้างข้อมูล

ข้อความ	คำตอบ	เหตุผล
([])	YES	ปิดวงเล็บชั้นในก่อนชั้นนอก
([)]	NO	ชนิดที่ปิดไม่ตรงกับชั้นล่าสุด
) (	NO	มีวงเล็บปิดก่อนมีวงเล็บเปิด
((	NO	ยังมีวงเล็บเปิดที่ไม่มีคู่



ก่อนจบวันนี้ ลองอธิบาย...

เมื่อสั่ง $p = p \rightarrow \text{next}$ เราเปลี่ยนตัวเดินหรือเปลี่ยนลิงก์ของโหนด และเหตุใดเราจึงควรเก็บ head ไว้

top() ต่างจาก pop() อย่างไร งานแบบใดเหมาะกับ queue และเหตุใด union จึงต้องรู้ว่าสมาชิกใดกำลังใช้งาน

สุดท้าย การใช้ผลของการอ่านข้อมูลเป็นเงื่อนไข while ช่วยให้เราอ่านไฟล์ได้ถูกต้องอย่างไร



สิ่งที่นำไปใช้ได้

- เราใช้ struct รวมข้อมูลที่เกี่ยวข้อง
- ใช้ pointer เชื่อมโหนด
- ใช้ stack หรือ queue จัดลำดับการทำงาน



เอกสารอ้างอิง

syllabus-toi-2569.pdf · ฉบับปรับปรุง 26 มิถุนายน 2569

eel.is/c++draft/stack.defn และ queue.defn

eel.is/c++draft/class.union และ filebuf.members

learn.microsoft.com · C++ unions และ enumerations

GeeksforGeeks - Data Structures <https://www.geeksforgeeks.org/data-structures/>

GeeksforGeeks - Linked List in C++ <https://www.geeksforgeeks.org/linked-list-set-1-introduction/>

GeeksforGeeks - Stack in C++ <https://www.geeksforgeeks.org/stack-data-structure-introduction-program/>

GeeksforGeeks - Queue in C++ <https://www.geeksforgeeks.org/queue-cpp-stl/>

เอกสารการสอน อ.ธารรัตน์ พวงสุวรรณ มหาวิทยาลัยบูรพา เรื่อง Data Structures and Algorithms

Mikelopster <https://docs.mikelopster.dev/>

<https://preecha11th.wordpress.com/>

cplusplus.com <http://www.cplusplus.com/>

Learncpp.com <https://www.learncpp.com/>

Stack Overflow <https://stackoverflow.com/>

Tutorialspoint <https://www.tutorialspoint.com/cplusplus/index.htm>

Programiz - C++ Programming <https://www.programiz.com/cpp-programming>

Cppreference.com <https://en.cppreference.com/w/>